

K8s 기본

inky4832@daum.net

1. Kubernetes 개요

1.1 Kubernetes

■ 개념

- 오픈 소스 시스템으로서 컨테이너 배포 및 관리 시스템의 표준
- <https://kubernetes.io/ko/>

■ 특징

- 어플리케이션 부하에 따른 자동 배포 기능
- 쉬운 Scaling 과 Load balancing
- 컨테이너 간의 원활한 통신을 위해 자동으로 네트워크를 구성
- 문제 발생 시 컨테이너 자동 재시작 및 재배포
- 필요시 이전 버전으로 롤백
- 표준이기 때문에 어떤 클라우드 공급자와도 연동이 가능



1.2 Kubernetes 구성

■ K8s Architecture

- K8s Cluster = Control Plane + Worker Nodes



1.2 Kubernetes 구성

■ 클러스터 (Cluster)

- 개념
 - K8s가 컨테이너를 운영하기 위해 모아둔 서버들의 집합
 - 즉 여러 대의 컴퓨터(노드)를 하나의 큰 시스템처럼 사용하는 환경
- 필요성
 - 서버 1대로 운영 시
 - 서버에 장애가 발생되면 서비스도 같이 영향을 받음
 - 트래픽 증가 시 확장 어려움
 - 클러스터 운영 시
 - 여러 노드에 분산 배치
 - 장애 발생 시 다른 노드로 재배포
 - 필요하면 노드 추가로 확장
- 구성
 - **Control Plane** : 감독/관리자 서버 (마스터 노드라고 부름)
 - **Worker Nodes** : 실제 컨테이너를 실행하는 서버

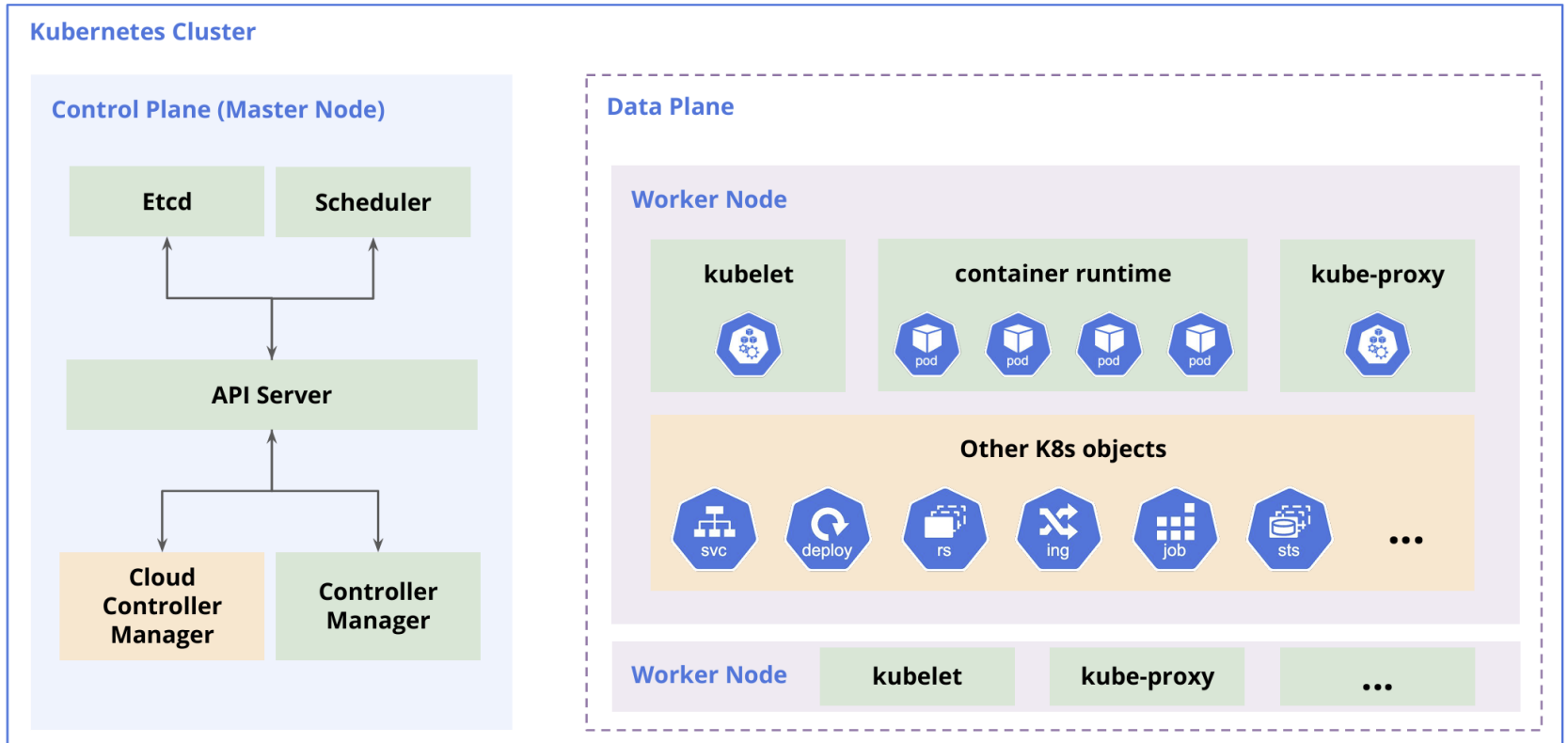
1.2 Kubernetes 구성

■ 노드 (Node)

- 개념
 - 클러스터를 구성하는 한 대의 컴퓨팅 자원(CPU/메모리/네트워크/디스크)을 가진 단위
- 종류
 - **Control Plane 노드 (master node)**
 - 클러스터 전체를 관리하는 구성요소들 포함
 - 주요 구성요소
 - » kube-apiserver : kubelet 과의 통신용
 - » scheduler : Pod를 어느 노드에 올릴지 결정
 - » controller-manager: Worker Node 자동 복구 담당 (Pod 개수 등)
 - » etcd: 클러스터 상태 저장소
 - **Worker 노드**
 - 실제 Pod(컨테이너)를 실행하는 곳
 - 주요 구성요소
 - » kubelet : master node와 worker node 간 통신 담당
 - » container-runtime: 컨테이너를 실제로 실행 (containerd 등)
 - » kube-proxy : 노드와 Pod 네트워크 통신 관리 담당

1.2 Kubernetes 구성

■ 노드 (Node) 요소

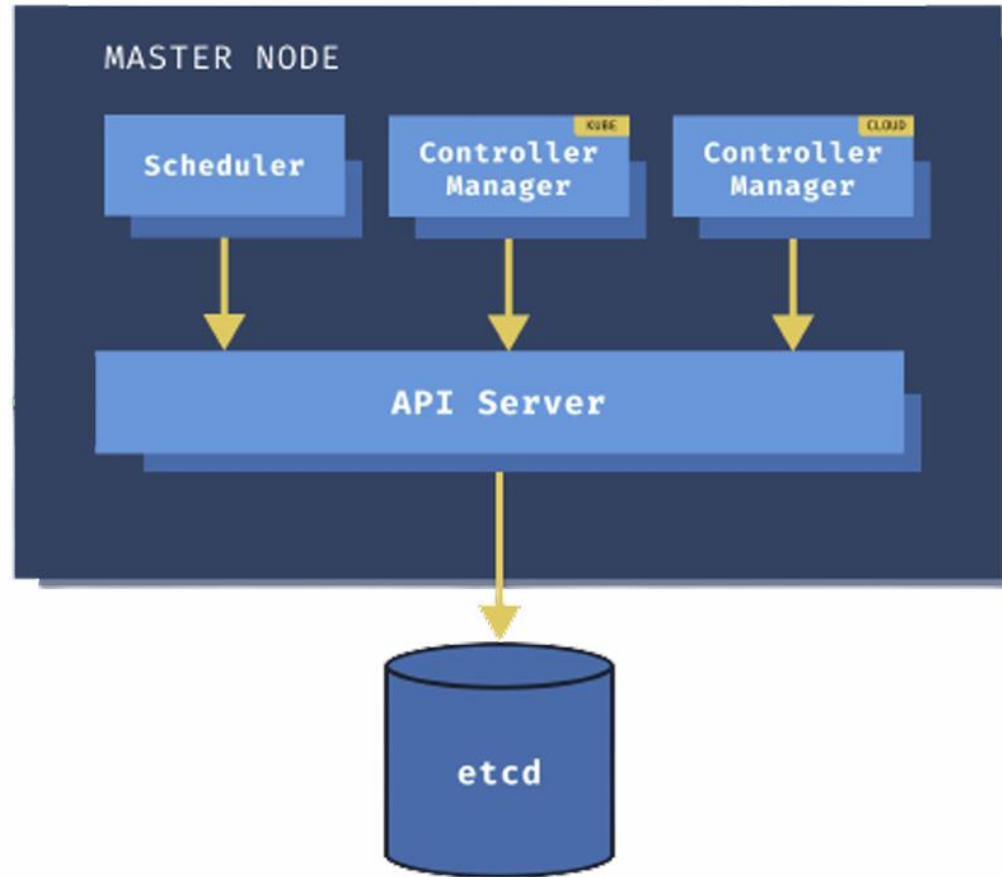


1.2 Kubernetes 구성

■ Control Plane (마스터 노드)

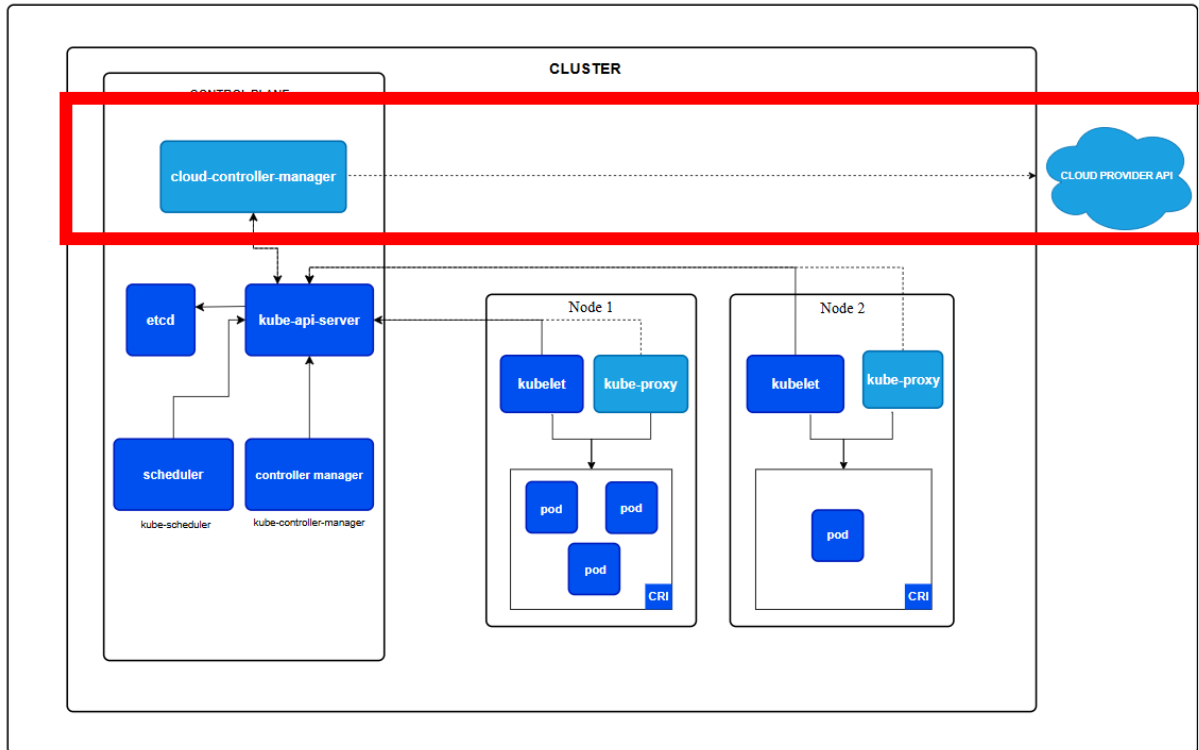
- 개념
 - 클러스터 전체를 관리하는 구성요소들을 포함한 한 대의 서버
- 구성요소
 - **API Server**
 - K8s API를 외부에 노출하는 구성요소로서 모든 관리 작업의 진입점(Entry Point)역할을 담당
 - kubectl, K8s Dashboard 등 모든 클라이언트 요청은 반드시 API Server를 통해 전달됨
 - **Scheduler**
 - Pod를 어떤 노드(Node)에 배치할 지 결정하는 역할을 담당
 - 노드의 CPU, 메모리 같은 자원 사용량과 제약조건 등을 고려하여 가장 적합한 노드를 선택함
 - **Kube Controller Manager**
 - 클러스터의 상태를 원하는 상태(desired state)로 유지하기 위해 여러 컨트롤러 프로세스들을 실행
 - ex. ReplicaSet Controller (Pod 개수 유지), Node Controller(노드 상태 관리), Service Controller (namespace 관리)
 - **Etcd**
 - 클러스터의 모든 **설정 정보와 상태 정보**를 저장하는 분산 Key-Value 저장소
 - ex. 모든 리소스 정보 (Pod, Service, ConfigMap 등), 클러스터 상태 정보 등

1.2 Kubernetes 구성



1.2 Kubernetes 구성

- **Cloud Controller Manager**
 - K8s 가 클라우드 환경(AWS, GCP, Azure 등)과 연동될 수 있도록 도와주는 구성요소
 - 주요 역할
 - » 클라우드 로드밸런서 관리
 - » 클라우드 기반 스토리지(PV) 연동
 - » 클라우드 노드 생성/삭제 관리



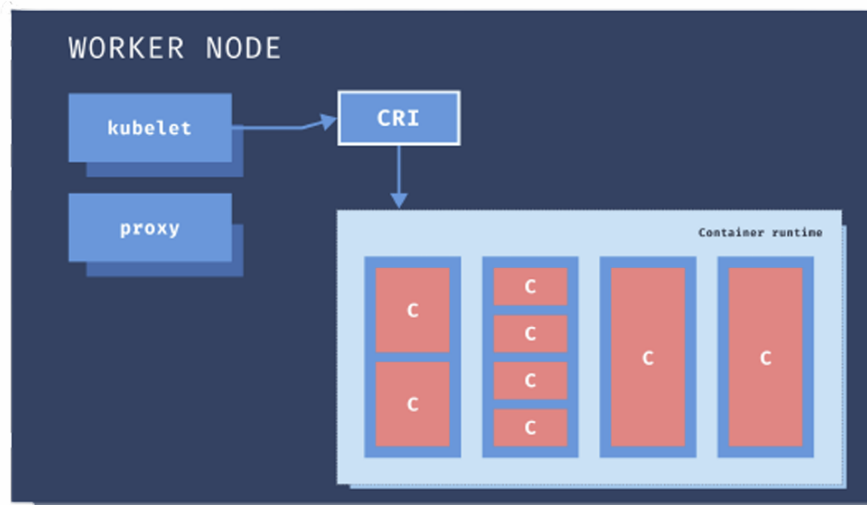
1.2 Kubernetes 구성

■ Worker 노드

- 개념
 - 실제 컨테이너를 포함하는 Pod가 실행되는 곳
- 구성요소
 - **kubelet**
 - 각 worker 노드에서 실행되는 에이전트(agent)로서 Control Plane과 통신하는 역할 담당
 - 주요기능
 - » API Server로부터 Pod 스펙(Pod 정의 정보)을 전달 받음
 - » 컨테이너가 Pod 안에서 정상적으로 실행되고 있는지 확인
 - » 컨테이너 상태를 Control Plane에 보고
 - **Container Runtime**
 - 각 worker 노드에서 컨테이너를 실제로 실행하는 런타임
 - 주요 역할
 - » 컨테이너 생성, 실행, 종료
 - » 이미지 다운로드 및 관리
 - » 리소스 관리 (CPU, 메모리 등)

1.2 Kubernetes 구성

- **kube-proxy**
 - 각 worker 노드에서 네트워크 규칙을 관리하는 컴포넌트
 - 주요기능
 - » Service 와 Pod 연결
 - » 로드밸런싱 수행
 - » iptables / IPVS 규칙 설정



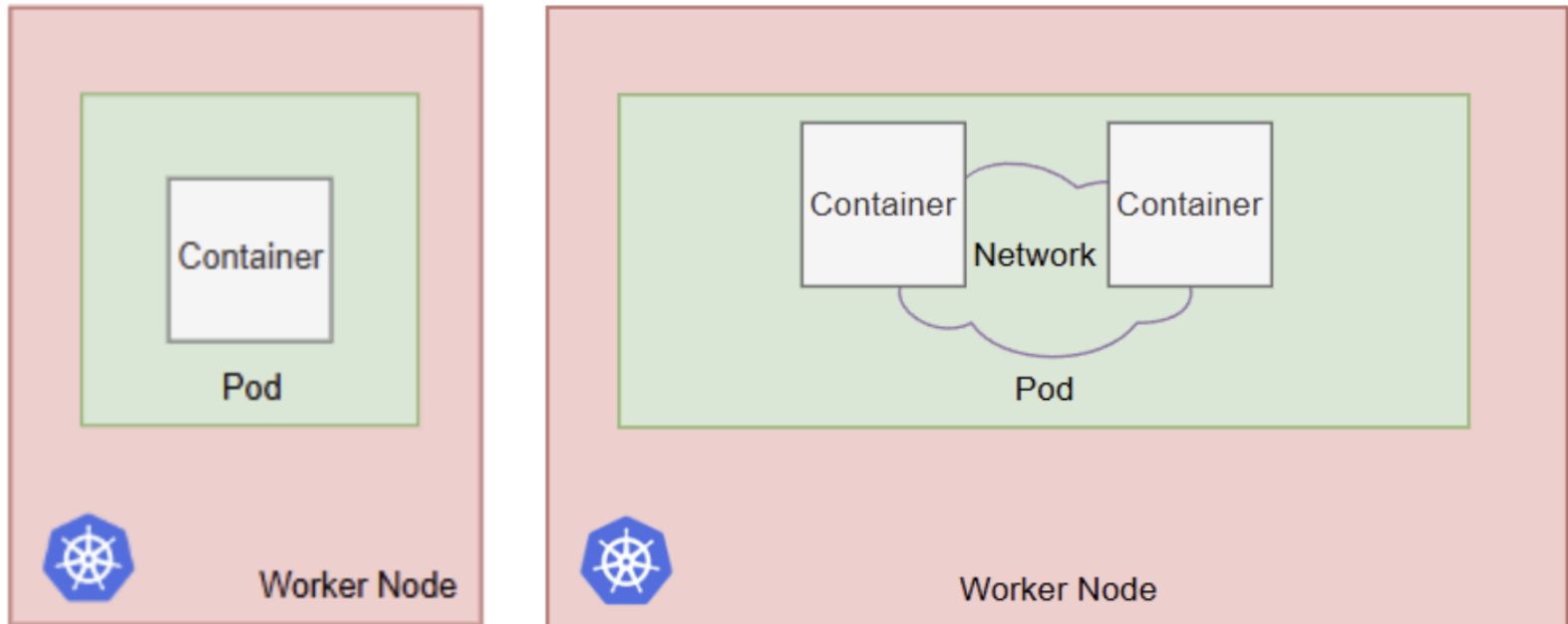
1.3 Kubernetes 핵심 객체

■ 파드 (Pod)

- 개념
 - K8s에서 컨테이너를 실행하는 최소 단위
- Pod가 필요한 이유
 - 컨테이너 개념만으로는 다음과 같은 동작을 '묶음 단위'로 운영하기가 어려움
 - 같이 실행해야 하는 프로세스 (메인 앱 + 보조 프로세스)
 - 같은 네트워크/스토리지 환경 공유
 - 하나의 '서비스 인스턴스'로서의 생명주기 관리
 - 즉 Pod는 '컨테이너와 공유 실행 환경'을 묶어 논리적으로 하나의 어플리케이션 인스턴스로 만드는 단위임
- Pod 주요 특징
 - Pod는 1개 이상 컨테이너를 포함한다.
 - Pod 내부 컨테이너 간에는 네트워크를 공유한다.
 - » 같은 Pod IP
 - » 그래서 같은 내부 컨테이너 간에는 localhost로 통신 가능
 - Pod 내부 컨테이너는 스토리지를 공유할 수 있다.
 - Pod는 '단일 인스턴스'로 동작됨
 - Pod는 일회성으로서 필요시 새로운 Pod로 바뀜.
 - Pod는 외부에서 직접 접근이 불가능하다.

1.3 Kubernetes 핵심 객체

- 파드 (Pod)

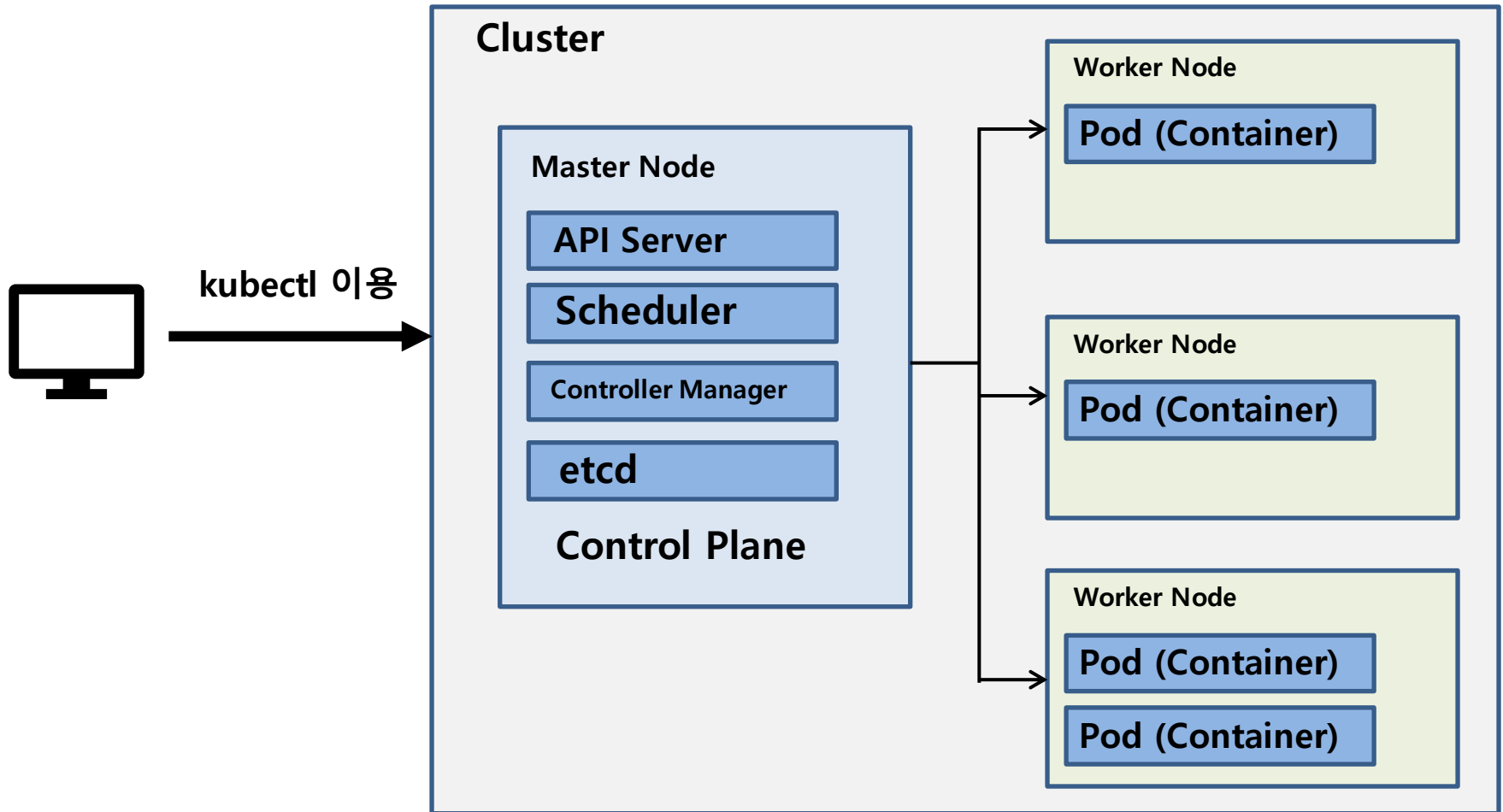


1.3 Kubernetes 핵심 객체

■ 서비스 (Service)

- 개념
 - 외부에서 Pod들을 접속할 수 있도록 기능을 제공하는 서비스
- 필요한 이유 2가지
 - 1) Pod는 자체적인 내부 IP를 가지고 있으나 외부에서 Pod를 접근하는데 사용 불가
 - 2) Pod는 제거되었다가 다시 생성되면 내부 IP가 새롭게 바뀜
 - 따라서 항상 고정된 접근방식이 필요함 (Service가 기능 제공)
- 주요 기능
 - 고정 IP/고정 DNS 제공
 - 여러 Pod로 트래픽 분산 (로드밸런싱)
 - 클러스터 내부 통신 표준 방식으로 처리
- Service 타입 종류
 - **ClusterIP**
 - 클러스터 내부에서만 접근 가능한 서비스
 - **NodePort**
 - 노드의 특정 포트를 열어서 외부에서 접근 가능
 - **LoadBalancer**
 - 외부에서 접근 가능하며 로드밸런싱 기능 제공

1.4 Kubernetes 아키텍처



2. Kubernetes 환경 설정

2.1 윈도우용 Kubernetes 설치

■ Docker Desktop Kubernetes

- Docker Desktop안에 미리 설정된 Kubernetes 클러스터를 자동으로 실행시켜주는 기능
- Docker Desktop에서 활성화 버튼 하나로 Kubernetes 바로 사용 가능

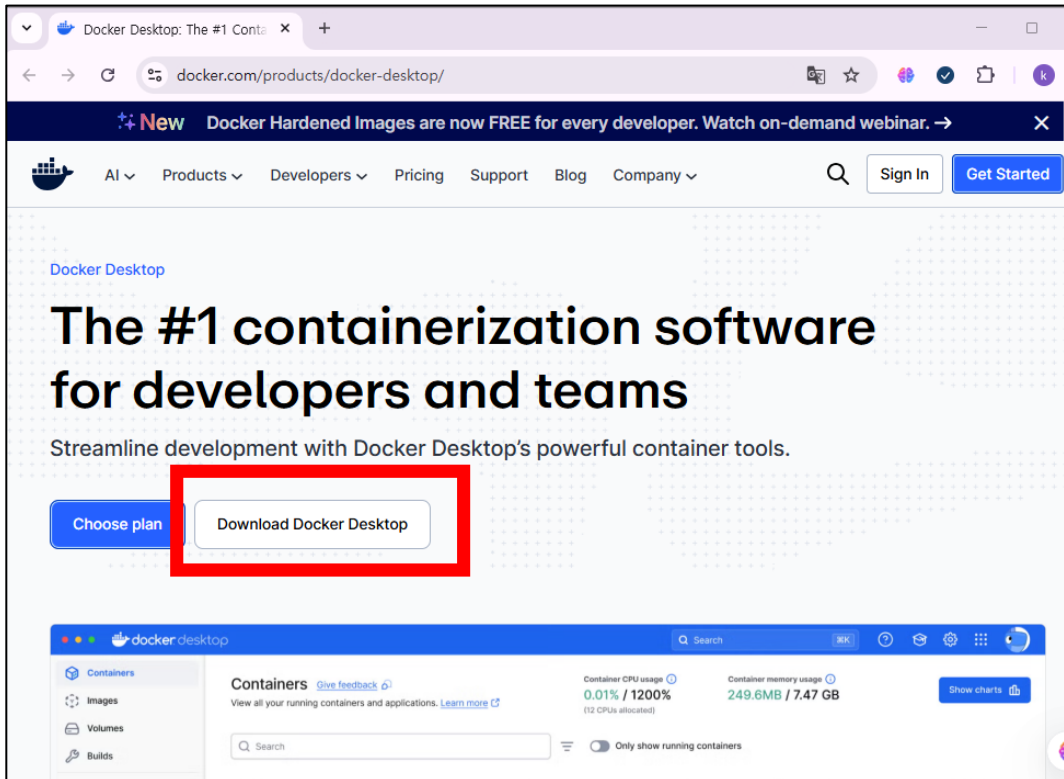
The screenshot shows the Docker pricing page with a navigation bar at the top containing links for AI, Products, Developers, Pricing, Support, Blog, and Company. There are also search, sign in, and get started buttons. Below the navigation bar, there are tabs for 'Yearly' and 'Monthly' pricing. The main content area displays four pricing plans:

Plan	Price	Target Audience	Key Features
Docker Personal	\$0	For individual developers who need the essential tools to build and deploy containers.	Docker Desktop, Docker Engine + Kubernetes, Docker Hub, Docker Scout, Docker Debug
Docker Pro	\$11 per user/month	For individual professionals who require more advanced features and additional resources.	Docker Build Cloud, Testcontainers Cloud, Synchronized File Shares, Visibility into Docker Scout health scores, 5 business day support response
Docker Team	\$16 per user/month	For small teams that need collaborative tools to make working together more efficient.	Add users in bulk, Audit logs, Docker Hub role-based access control, 2 business day support response
Docker Business	\$24 per user/month	For enterprises desiring robust security, control, and compliance features.	Hardened Docker Desktop, Single Sign-On (SSO), SCIM user provisioning, Image and Registry Access Management, Desktop Insights Dashboard

2.1 윈도우용 Kubernetes 설치

■ Docker Desktop 다운로드 및 설치

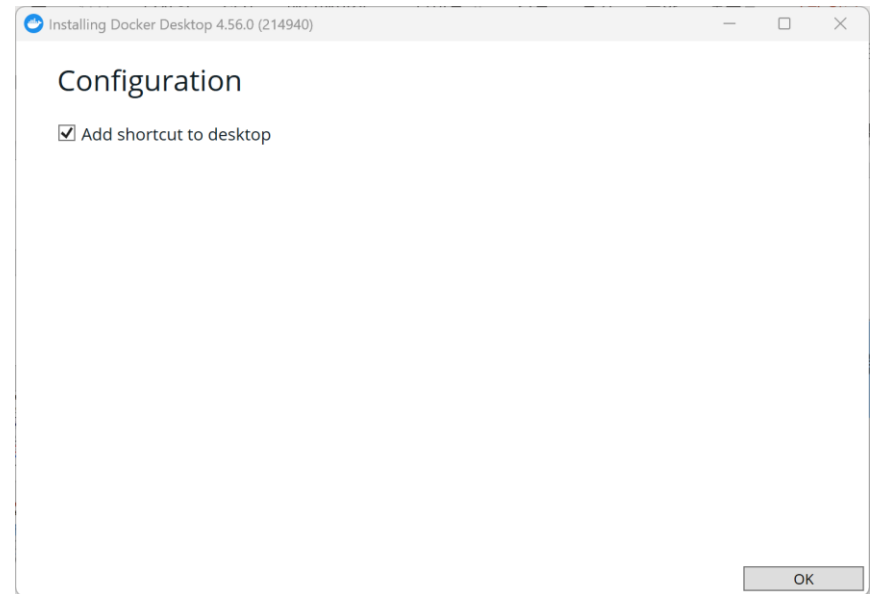
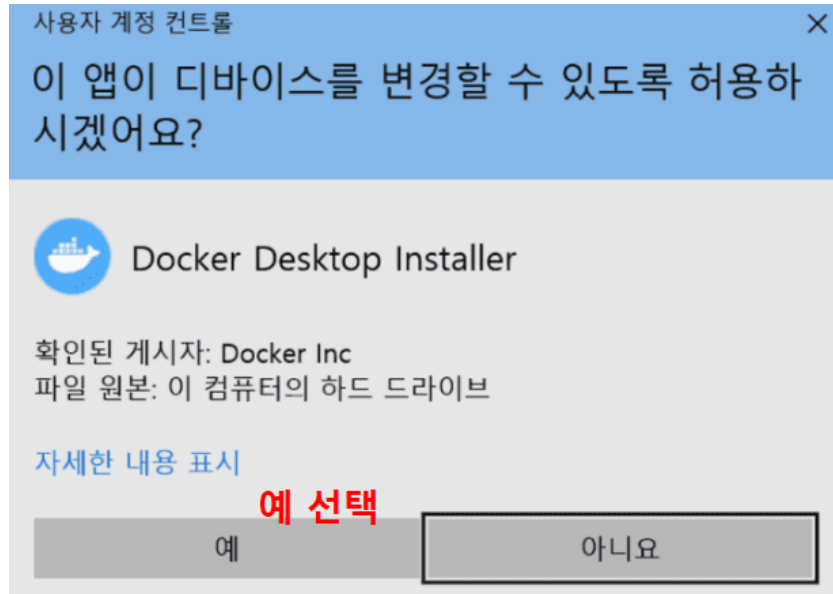
- <https://www.docker.com/products/docker-desktop/>



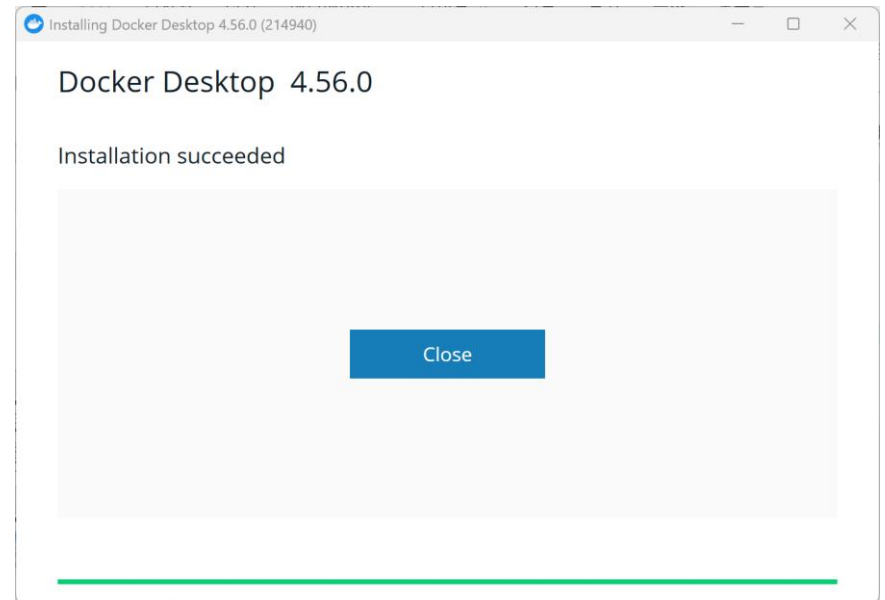
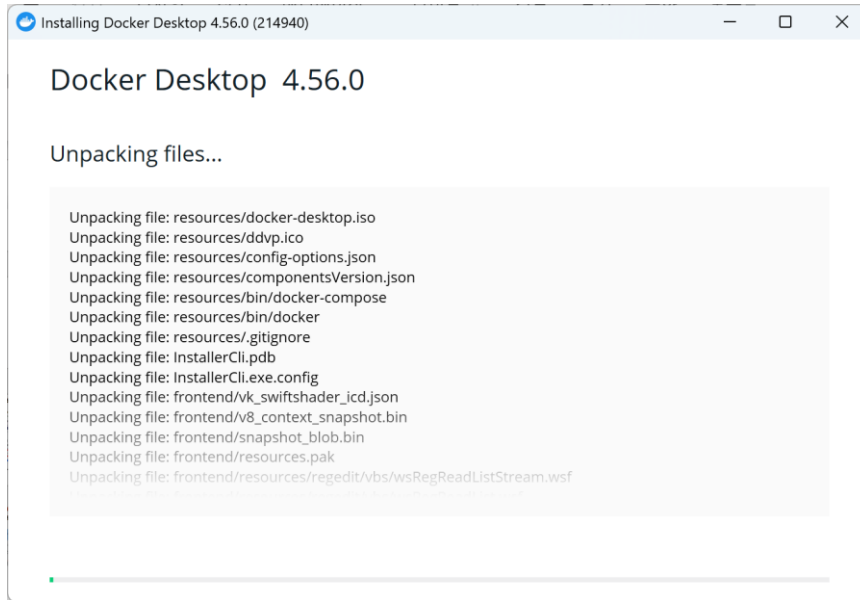
■ Docker 설치 문서

- <https://docs.docker.com/desktop/setup/install/windows-install/>
- <https://docs.docker.com/desktop/setup/install/mac-install/>

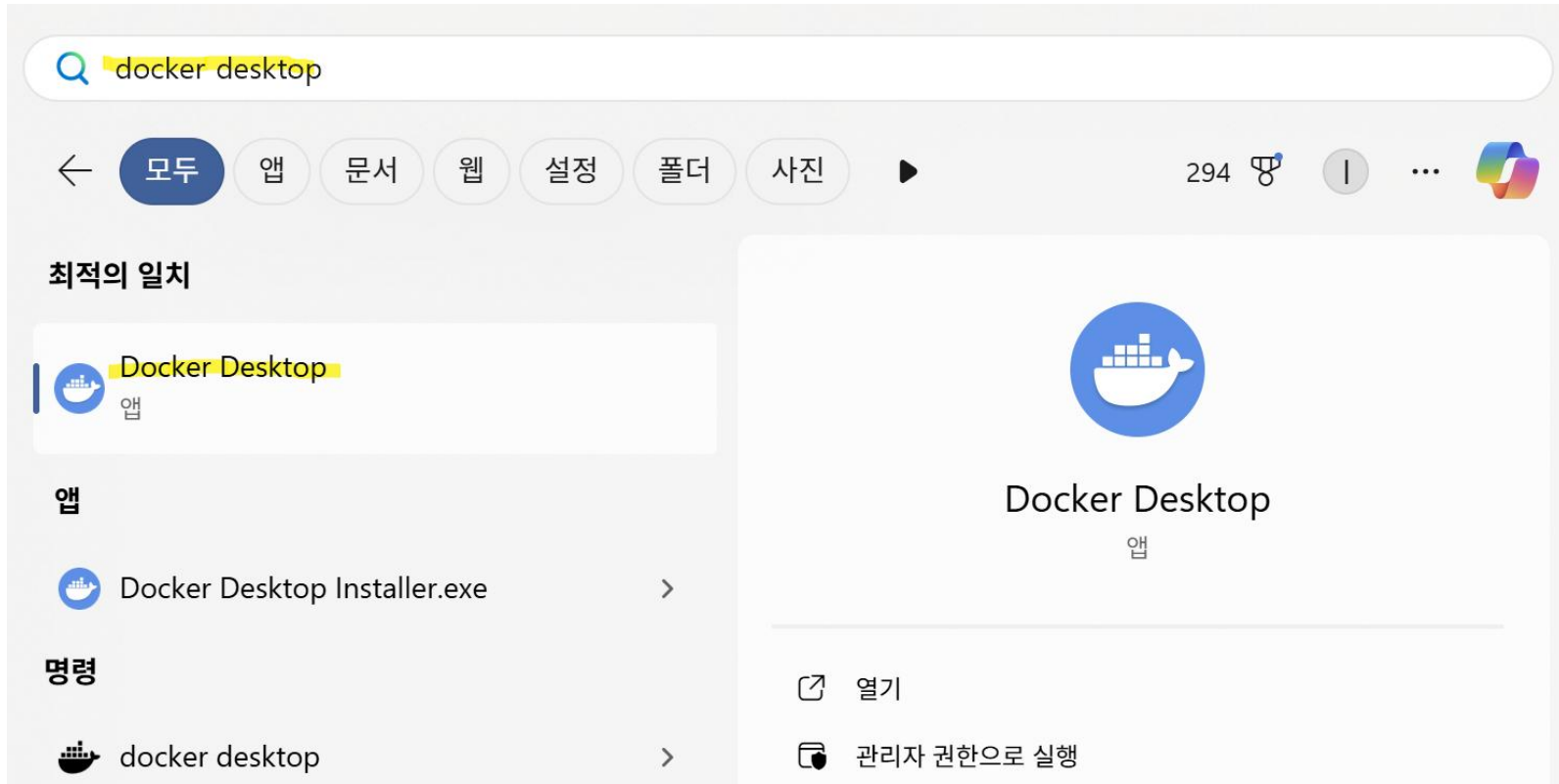
2.1 윈도우용 Kubernetes 설치



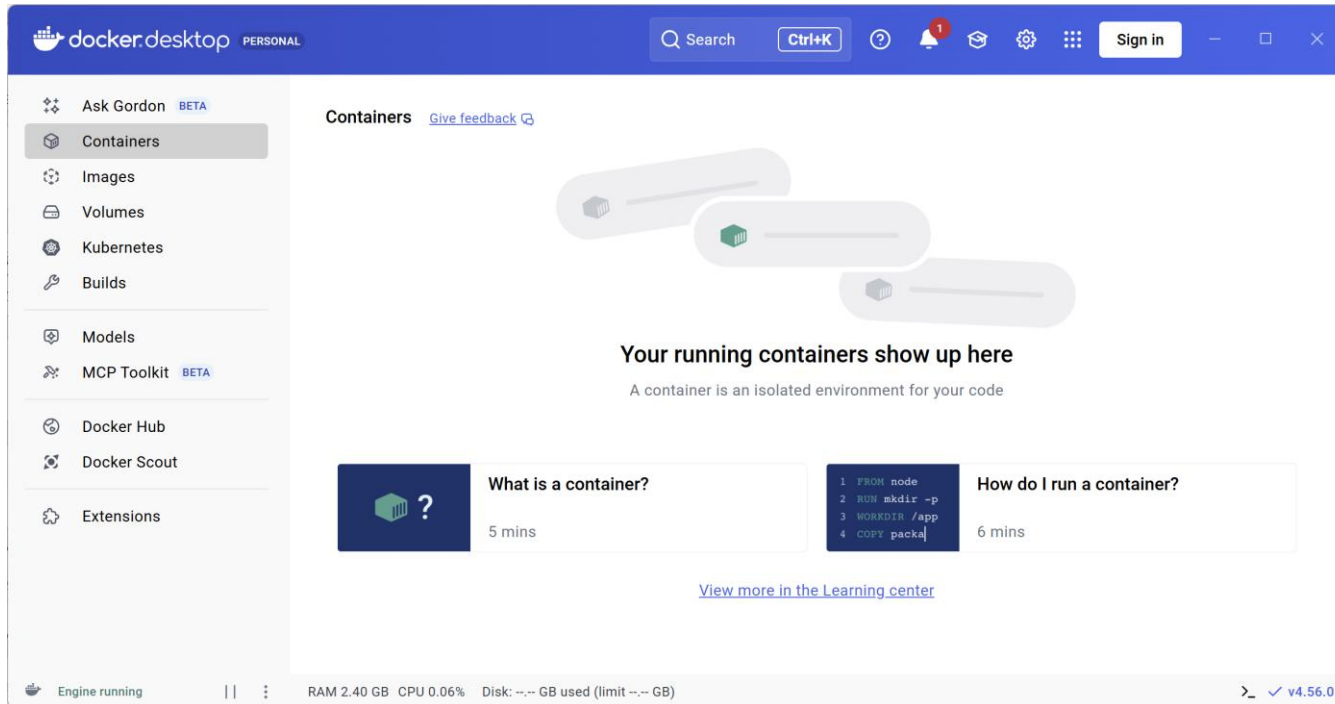
2.1 윈도우용 Kubernetes 설치



2.1 윈도우용 Kubernetes 설치



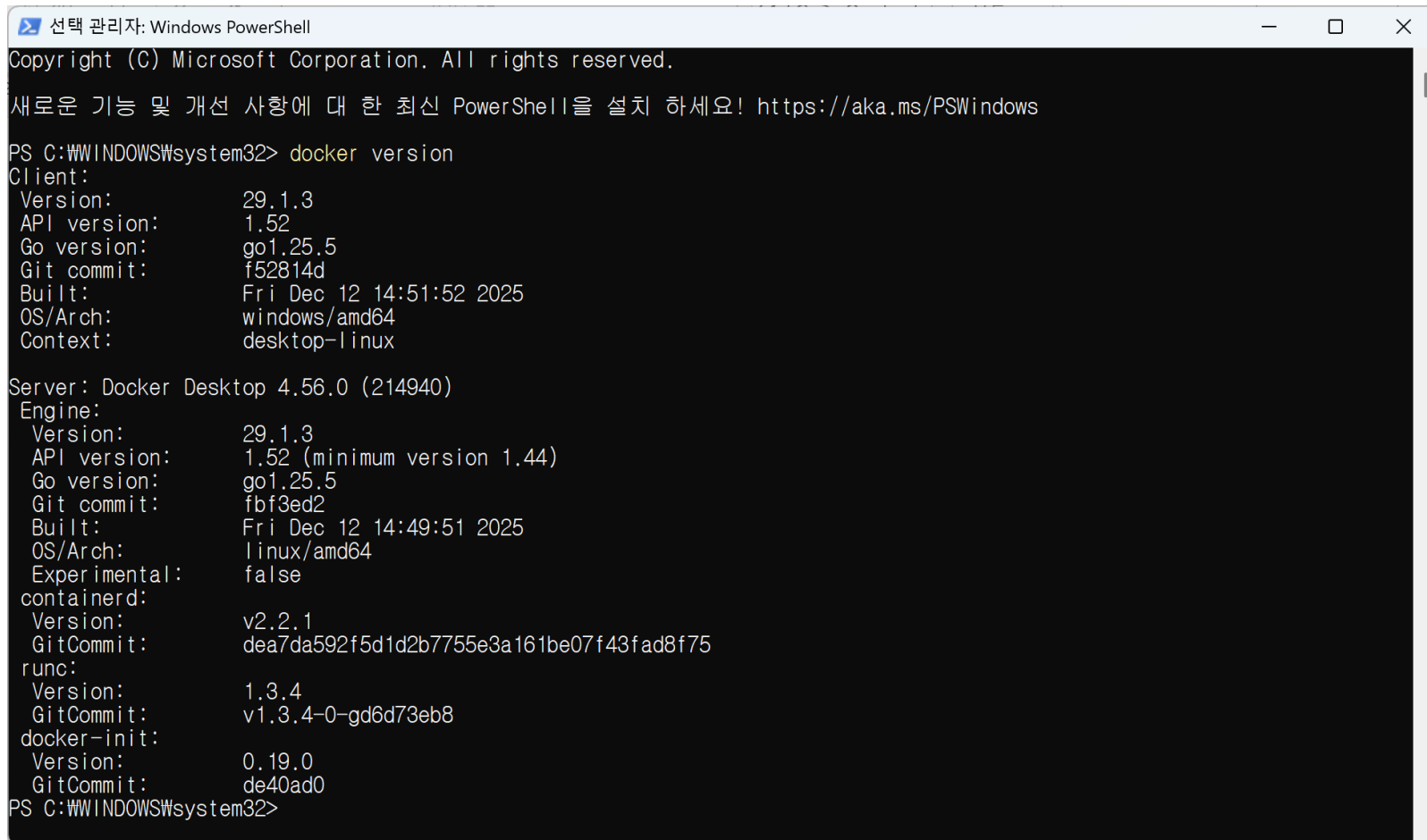
2.1 윈도우용 Kubernetes 설치



2.1 윈도우용 Kubernetes 설치

■ Docker 버전 확인

- Window PowerShell 이용한다.
- 명령어: `docker version`



```
선택 관리자: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대 한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows

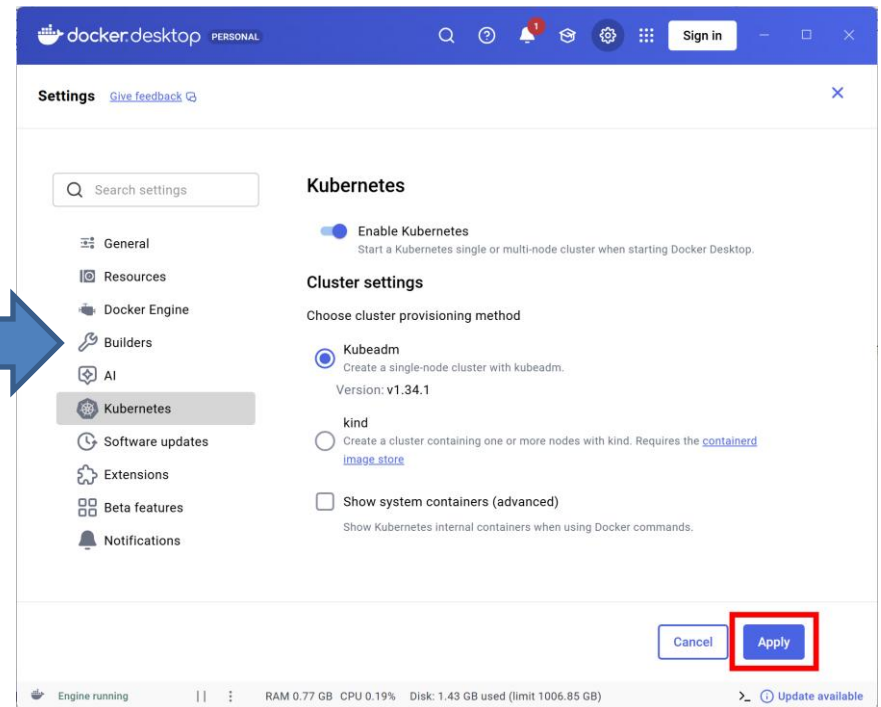
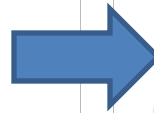
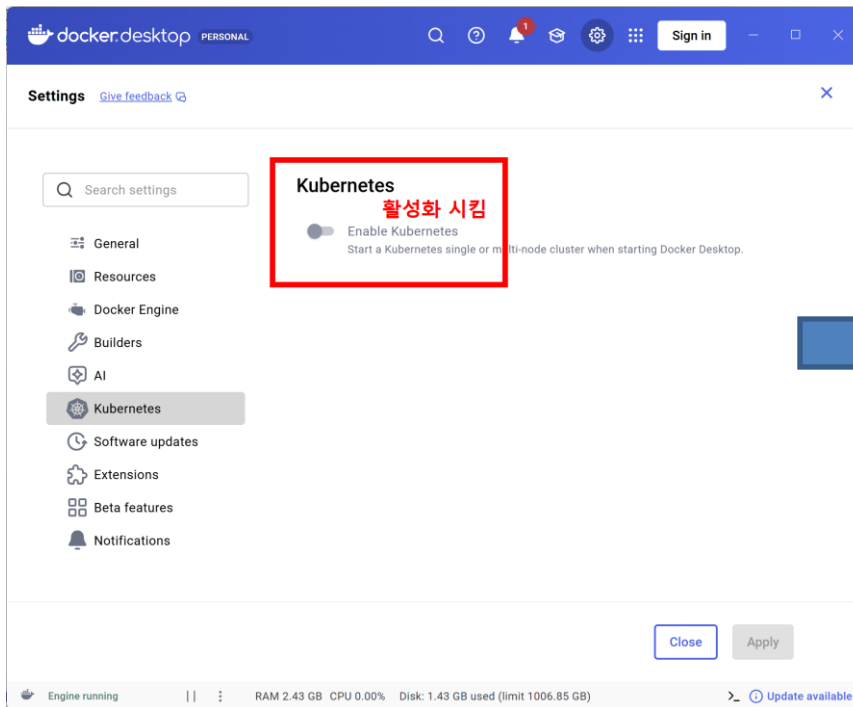
PS C:\WINDOWS\system32> docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.25.5
 Git commit:        f52814d
 Built:             Fri Dec 12 14:51:52 2025
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.56.0 (214940)
Engine:
 Version:           29.1.3
 API version:       1.52 (minimum version 1.44)
 Go version:        go1.25.5
 Git commit:        fbf3ed2
 Built:             Fri Dec 12 14:49:51 2025
 OS/Arch:           linux/amd64
 Experimental:      false
 containerd:
 Version:           v2.2.1
 GitCommit:         dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
 Version:           1.3.4
 GitCommit:         v1.3.4-0-gd6d73eb8
 docker-init:
 Version:           0.19.0
 GitCommit:         de40ad0
PS C:\WINDOWS\system32>
```

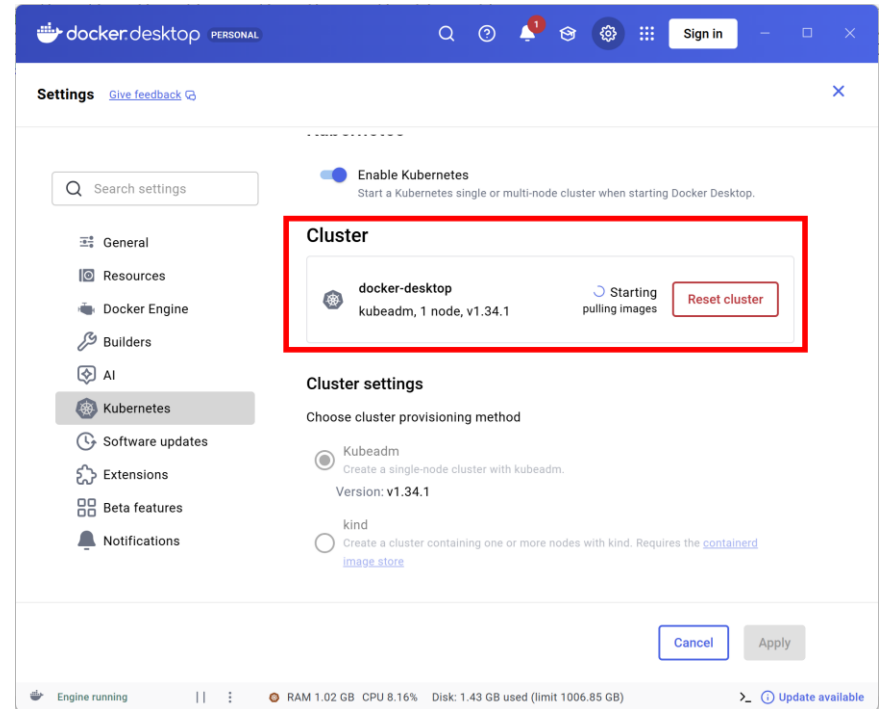
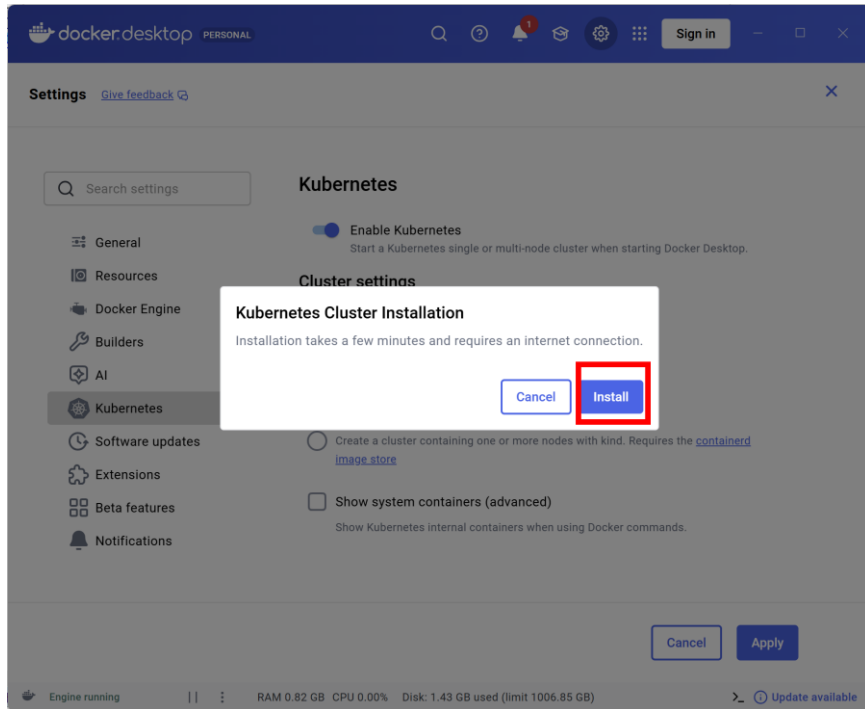

2.1 윈도우용 Kubernetes 설치

■ Docker Desktop Kubernetes 활성화 방법

- 1. Docker Desktop 실행
- 2. Settings 선택하고 Kubernetes 클릭
- 3. Enable Kubernetes 선택 > Kubeadm 선택 (Single-node)
- 4. Apply & Kubernetes Cluster Installation

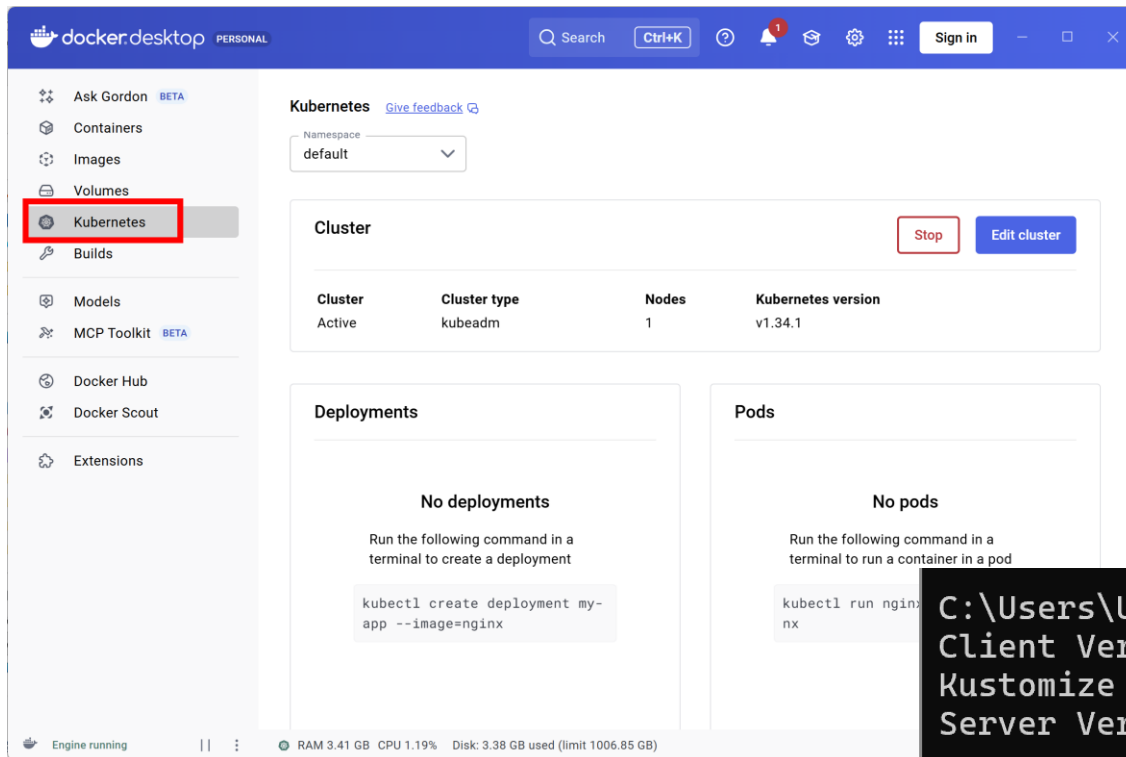


2.1 윈도우용 Kubernetes 설치



2.1 윈도우용 Kubernetes 설치

- 설치 확인
 - kubectl version



```
C:\Users\User>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.1
```

2.1 윈도우용 Kubernetes 설치

- 추가로 다음과 같이 Kubernetes 관련 Image들이 설치됨 (삭제 불가)

The screenshot shows the Docker Desktop 'Images' tab. A red rectangle highlights a list of 12 images installed locally. The images are:

Name	Tag	Image ID	Created	Size
docker/desktop-kube	kubernetes-v1.34.1-c	12d6673564e0	4 months ago	591.99 MB
registry.k8s.io/kube-	v1.34.1	6e9fbc4e25a5	4 months ago	73.5 MB
registry.k8s.io/kube-	v1.34.1	2bf47c1b01f5	4 months ago	101.09 MB
registry.k8s.io/kube-	v1.34.1	b9d7c117f8ac	4 months ago	118.37 MB
registry.k8s.io/kube-	v1.34.1	913cc83ca0b5	4 months ago	101.65 MB
registry.k8s.io/etcd	3.6.4-0	e36c08168342	6 months ago	272.54 MB
registry.k8s.io/pause	3.10.1	278fb9dbcca9	7 months ago	1.05 MB
registry.k8s.io/cored	v1.12.1	e8c262566636	10 months ago	100.71 MB
registry.k8s.io/pause	3.10	ee6521f290b2	2 years ago	1.05 MB
docker/desktop-vpnk	dc331cb22850be0cc	7ecf567ea070	3 years ago	47 MB
docker/desktop-stor	v2.0	115d77efe6e2	5 years ago	59.16 MB
kubernetesui/metric	v1.0.8	76049887f07a	4 years ago	63.58 MB

The interface also shows a search bar, a 'Sign in' button, and a status bar at the bottom indicating 'Engine running' and system resources (RAM 2.26 GB, CPU 1.19%, Disk 3.75 GB used).

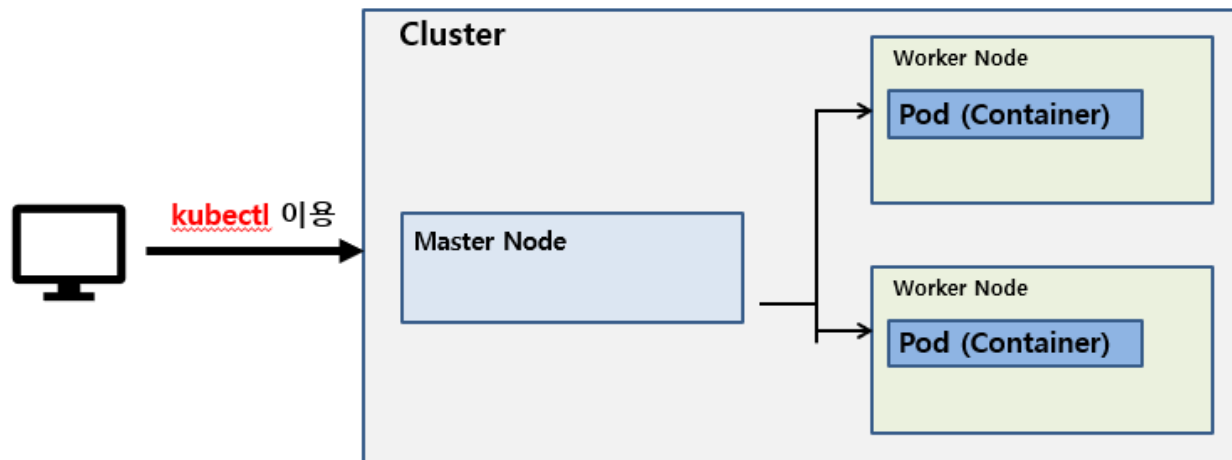
2.2 kubectl

■ 개념

- K8s 클러스터를 제어하는 공식 CLI 도구임
- 즉 K8s API Server에 명령을 보내는 도구

■ 주요 동작

- 리소스 생성/조회/수정/삭제
- Pod 상태 확인
- 로그 확인
- 컨테이너 내부 접속
- 배포 관리



2.2 kubectl

■ 문법

- `kubectl [command] [type] [name] [flags]`
- <https://kubernetes.io/docs/reference/kubectl/>

■ 명령어 구성요소

구분	설명	특징/규칙	예
command	리소스에 대해 수행할 동작 (Operation)	동사 역할 여러 리소스에 적용 가능	<code>kubectl get pods</code> <code>kubectl delete pod nginx</code>
type	대상이 되는 리소스 종류	대소문자 구분 안함 단수/복수/약어 모두 가능	<code>kubectl get pod</code> <code>kubectl get pods</code> <code>kubectl get po</code>
name	특정 리소스의 이름	선택사항 생략 시 해당 타입 전체 적용	<code>kubectl get pods</code> <code>kubectl get pod nginx</code>
flags	명령어 동작을 제어하는 옵션 (Option)	선택사항 출력, 범위, 방식 제어	<code>kubectl get pods -n dev</code> <code>kubectl get pod nginx -o yaml</code>

2.2 kubectl

■ [command] 종류

- 리소스에 대해서 무엇을 할 것인가를 지정

command	설명	예시
get	리소스 조회	<code>kubectl get pods</code>
describe	상세 정보 조회	<code>kubectl describe pod nginx</code>
apply	생성/변경	<code>kubectl apply -f app.yml</code>
delete	리소스 삭제	<code>kubectl delete deployment webapp</code>
logs	로그 확인	<code>kubectl logs pod-name</code>
exec	컨테이너 접속	<code>kubectl exec -it pod-name -- sh</code>
scale	개수 조정	<code>kubectl scale deployment webapp --replicas=3</code>
rollout	배포 관리	<code>kubectl rollout status deployment/webapp</code> <code>kubectl rollout history deployment/webapp</code> <code>kubectl rollout undo deployment/webapp</code> <code>kubectl rollout restart deployment/webapp</code>
config	클러스터 접속 관리	<code>kubectl config current-context</code> <code>kubectl config get-contexts</code> <code>kubectl config use-context 컨텍스트</code> <code>kubectl config delete-context 컨텍스트</code>

2.2 kubectl

■ [type] 종류

- 대상이 되는 리소스 지정

type	설명	약어	예시
Pod	컨테이너 실행 단위	po	kubectl get pod
Deployment	Pod 관리 (배포)	deploy	kubectl get deploy
Service	Pod 접근	svc	kubectl get svc
ReplicaSet	Pod 복제관리	rs	kubectl get rs
Namespace	논리적 공간	ns	kubectl get ns
ConfigMap	환경변수 설정	cm	kubectl get cm
Secret	민감 정보	secret	kubectl get secret
PersistentVolume	스토리지	pv / pvc	kubectl get pv kubectl get pvc

2.2 kubectl

■ [flags] 종류

- 명령어 부분을 제어하는 옵션

flag	설명	예시
-n	네임스페이스 지정	kubectl get pods -n dev
-o wide	상세 출력	kubectl get pods -o wide
-o yaml	YAML 출력	kubectl get pod nginx -o yaml
-f	파일 지정	kubectl apply -f deploy.yaml
-l	라벨 셀렉터	kubectl get pods -l app=webapp
-it	인터랙티브 접속	kubectl exec -it pod -- bash
--all-namespaces	전체 네임스페이스 조회	kubectl get pods -A
--show-labels=true	labels 값 출력	kubectl get pod --show-labels=true

2.3 kubeconfig

■ 정의

- kubectl이 K8s 클러스터에 접근하기 위한 설정 파일

■ kubectl 이 반드시 알아야 될 정보

- 어느 클러스터인가? ex. Cloud, Local
- 누구 권한으로 접속하는가? ex. 사용자 인증 정보
- 기본 작업 공간은 어디인가? ex. namespace
- 이러한 정보를 포함하는 것이 kubeconfig 파일임
- 기본 저장 위치
 - C:\Users\<사용자명>\.kube\config

■ kubeconfig 특징

- 여러 클러스터 정보 저장 가능
- 여러 사용자 인증 정보 저장 가능
- 클러스터 + 사용자 조합을 context 라는 개념을 사용하여 관리함

2.3 kubeconfig

■ 주요 명령어

설명	예시
컨텍스트 전체 목록 확인	kubectl config get-contexts kubectl config view
현재 컨텍스트 확인	kubectl config current-context
컨텍스트 변경 명령어	kubectl config use-context <context-name>
컨텍스트 삭제	kubectl config delete-context <context-name>

```
Windows PowerShell
PS C:\Users\User> kubectl config get-contexts
CURRENT  NAME          CLUSTER          AUTHINFO          NAMESPACE
*         docker-desktop  docker-desktop  docker-desktop
PS C:\Users\User> kubectl config current-context
docker-desktop
PS C:\Users\User>
```

2.4 K8s Configuration / 구성방식 설정

■ 개념

- K8s 에서 클러스터의 원하는 상태 (Desired state)를 정의하고 관리하는 방식의미.

■ 설정 방법 2가지

- Imperative Configuration (명령형 구성)
 - 정의
 - 시스템에 단계별 명령을 직접 지시하는 방식이다.
 - 사람이 순서를 관리해야 한다.
 - ex.
 - » `docker run -d 80:80 nginx`
 - » `docker pull nginx`
 - 장점
 - » 즉각적/직관적
 - » 테스트나 단발성 작업에 편리
 - 단점
 - » 실행 이력이 남지 않음
 - » 동일 상태 재현이 어려움
 - » 장애 발생 시 자동 복구 불가
 - » 사람이 계속 관리해야 됨

2.4 K8s Configuration / 구성방식 설정

- Declarative Configuration (선언형 구성)
 - 정의
 - 클러스터의 원하는 상태 (Desired state)만 선언하면 시스템이 알아서 처리하는 방식
 - YAML 이용하여 선언
 - 핵심 상태 2가지
 - Desired State (원하는 상태)
 - » 사용자가 '최종적으로 이렇게 되어야 돼' 라고 선언하는 상태
 - » ex. Pod는 3개 (replica=3)
 - Actual State (실제 상태)
 - » 클러스터(시스템)가 지금 실제로 가지고 있는 상태
 - » ex. 실제 Pod가 2개만 실행되고 있음
 - Declarative의 핵심 원리
 - 시스템(Control plane)은 Actual State를 계속 감시하면서 Desired State와 다르면 자동으로 맞추기를 시도한다.
 - 장점
 - 자동 복구 (Self-healing)
 - 상태 유지 (Consistency)
 - 재현성 (Reproducibility)
 - 버전 관리/감사 (Audit)
 - 대규모 운영에 유리

2.5 K8s YAML

■ 개념

- K8s 의 클러스터가 원하는 상태(Desired State)를 갖도록 선언하는 파일.

■ 공통 기본구조

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  ...
```

주요 속성	설명
apiVersion	K8s API 버전
kind	리소스 종류
metadata	이름, 라벨 등 메타정보
spec	원하는 상태(Desired State) 정의

2.5 K8s YAML

■ apiVersion

- K8s 리소스마다 API 버전이 다름
- 틀리면 적용 자체가 안됨

리소스	apiVersion
Pod	v1
Service	v1
Deployment	apps/v1
ReplicaSet	apps/v1
ConfigMap	v1
Secret	v1
Namespace	v1
PersistentVolume	v1
PersistentVolumeClaim	v1
Ingress	networking.k8s.io/v1

2.5 K8s YAML

■ kind

- 생성하려는 K8s 리소스의 종류
- 예시

kind: Pod

kind: Deployment

kind: Service

```
C:\cloud_engineering\K8s>kubectl api-resources
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
componentstatuses	cs	v1	false	ComponentStatus
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pv	v1	false	PersistentVolume
Pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service
mutatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	MutatingWebhookConfiguration
validatingadmissionpolicies		admissionregistration.k8s.io/v1	false	ValidatingAdmissionPolicy
validatingadmissionpolicybindings		admissionregistration.k8s.io/v1	false	ValidatingAdmissionPolicyBinding
validatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	ValidatingWebhookConfiguration
customresourcedefinitions	crd,crds	apiextensions.k8s.io/v1	false	CustomResourceDefinition
apiservices		apiregistration.k8s.io/v1	false	APIService
controllerrevisions		apps/v1	true	ControllerRevision
daemonsets	ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true	Deployment
replicasets	rs	apps/v1	true	ReplicaSet

2.5 K8s YAML

■ metadata

- 이름, 네임스페이스, 라벨 등 리소스 식별 정보 지정
- 예시

```
metadata:  
  name: pod-nginx  
  namespace: dev  
  labels:  
    app: nginx
```

주요 속성	설명
name	리소스 이름 (필수)
namespace	네임스페이스
labels	리소스 분류용 키/값 Service 및 Deployment 연결 시 사용됨 (중요)

2.5 K8s YAML

■ spec

- spec은 리소스마다 완전히 다름
- 공통 개념은 동일 (Desired State 선언)
- 예시

```
apiVersion: v1
kind: Pod                # 리소스
metadata:
  name: pod-nginx        # pod 명
  labels:
    app: nginx           # 리소스 분류용 키/값
spec:
  containers:
    - name: nginx        # 컨테이너명
      image: nginx:1.26  # Docker 이미지명
  ports:
    - containerPort: 80  # 컨테이너 내부 80 포트 오픈 문서화
```

3. Kubernetes 주요 객체

3.1 Pod

■ 개념

- K8s에서 생성/관리할 수 있는 가장 작은 배포 단위임.
- <https://kubernetes.io/docs/concepts/workloads/pods/>

■ 주요 특징

- 가장 작은 배포/실행 단위 (장애/재시작/삭제/복제도 Pod 단위로 처리됨)
- 1 개 이상의 컨테이너 포함 가능
- Pod 내부 컨테이너들은 '네트워크를 공유'
- Pod 내부 컨테이너들은 '스토리지(Volume) 공유'
- Pod 는 LifeCycle를 가짐. (pending, running, succeeded, failed)

3.1 Pod

■ 장점

- 컨테이너 실행을 '표준화된 운영 단위'로 관리
 - 컨테이너 + 네트워크 + 스토리지 + 실행정책을 한 단위로 다루기 쉬움
- 멀티 컨테이너 조합으로 강력한 패턴 구현
 - sidecar 패턴으로 로그/모니터링/프록시를 앱과 '한 몸'처럼 운영 가능
 - 동일 네트워크(localhost)로 빠르고 단순한 통신 가능
- Self-healing 기반이 되는 단위
 - Pod가 죽거나 노드가 장애가 나면 Control Plane이 다시 Desired State를 맞추려고 함.
 - 이때 원하는 상태(Desired State)를 유지하기 위한 최소 단위가 Pod임

■ 단점

- Pod는 일회성이다. (Pod는 언제든지 죽고 다시 생성될 수 있음. 가변 IP)
- Pod를 직접 운영하면 관리가 어렵다.
- 멀티 컨테이너 Pod는 서로 간에 영향을 미친다.

3.1 Pod

■ ImagePullPolicy

- Pod가 시작될 때 컨테이너 이미지를 언제, 어떤 방식으로 가져올지 결정하는 정책이다.

■ 종류 3가지

- Always
 - Pod 시작할 때마다 Registry에 이미지 확인 요청.
 - 로컬에 동일한 digest 이미지가 있으면 캐시 사용하고 없으면 다운로드 됨.
- IfNotPresent
 - 노드에 이미지가 없을 때만 다운로드
- Never
 - 항상 로컬에서 이미지 사용

■ ImagePullBackOff

- K8s가 이미지를 가져오지 못해서 컨테이너를 시작하지 못하는 상태 의미.
- 주요원인: 이미지 및 태그명 오타, Docker Hub 접근 실패 등.

3.1 Pod

■ tag에 따른 기본 정책 자동화

- K8s는 tag에 따라서 기본 정책을 자동으로 결정한다.
 - tag가 latest 인 경우
 - 기본적으로 **Always**로 처리
 - tag가 특정 버전인 경우
 - 기본적으로 **IfNotPresent**로 처리

■ 기본 샘플

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-always
spec:
  containers:
    - name: nginx
      image: nginx:latest
      #imagePullPolicy: Always
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-cache
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      #imagePullPolicy: IfNotPresent
```

3.1 Pod

■ 주요 명령어

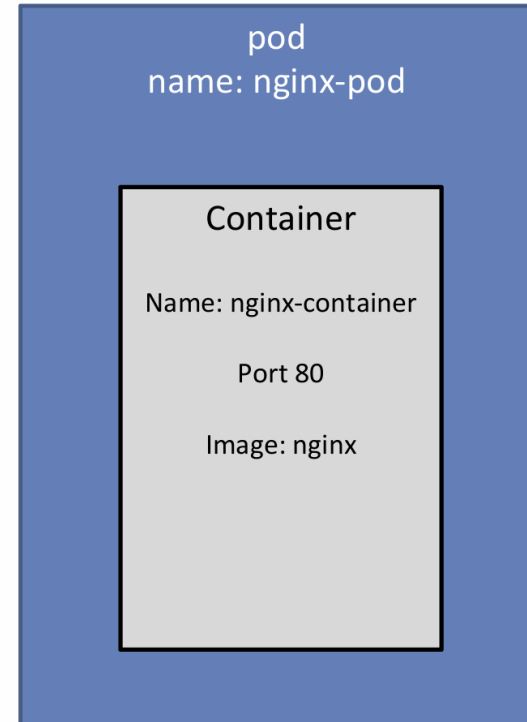
설명	예시
pod 생성/수정	kubectl apply -f 파일명.yaml
pod 조회	kubectl get pods kubectl get pod kubectl get pod -o wide #상세 정보 (IP, Node 등) kubectl get pod -w # 상태 실시간 모니터링
pod 상세 조회 확인	kubectl describe pod <pod명>
pod 로그 조회	kubectl logs <pod명>
pod 삭제	kubectl delete pod <pod명> kubectl delete -f 파일명.yaml
pod 설정 수정	kubectl edit pod <pod명>
CPU/메모리 사용량 조회	kubectl top pod
파일 복사	kubectl cp
포트 매핑 (포트 포워딩)	kubectl port-forward
로그 확인	kubectl logs kubectl logs -f

3.1 K8s 첫 번째 실습

■ 실습 순서

- 1. 임의의 프로젝트 생성 (empty project 선택)
- 2. nginx-pod.yaml 파일 생성
- 3. kubectl apply -f nginx-pod.yaml
- 4. kubectl get pods

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```



3.1 K8s 첫 번째 실습

■ 생성된 Pod에 접속하는 방법

▪ 1. Pod 내부에 직접 접근하기

- 문법: `kubectl exec -it [파드명] -- bash`
- ex.

```
kubectl exec -it nginx-pod -- bash
root@nginx-pod:/# curl http://localhost:80
```

▪ 2. Pod 내부 네트워크를 외부에서도 접속할 수 있도록 Port forwarding 하기

- 문법: `kubectl port-forward pod/[파드명] [로컬포트]:[파드포트]`
- ex.

```
kubectl port-forward pod/nginx-pod 80:80
웹 브라우저에서 http://localhost:80 요청
```

■ Pod 삭제하기

- 문법: `kubectl delete pod [파드명]`
- `kubectl delete pods --all`
- `kubectl delete -f nginx-pod.yaml`

▪ ex.

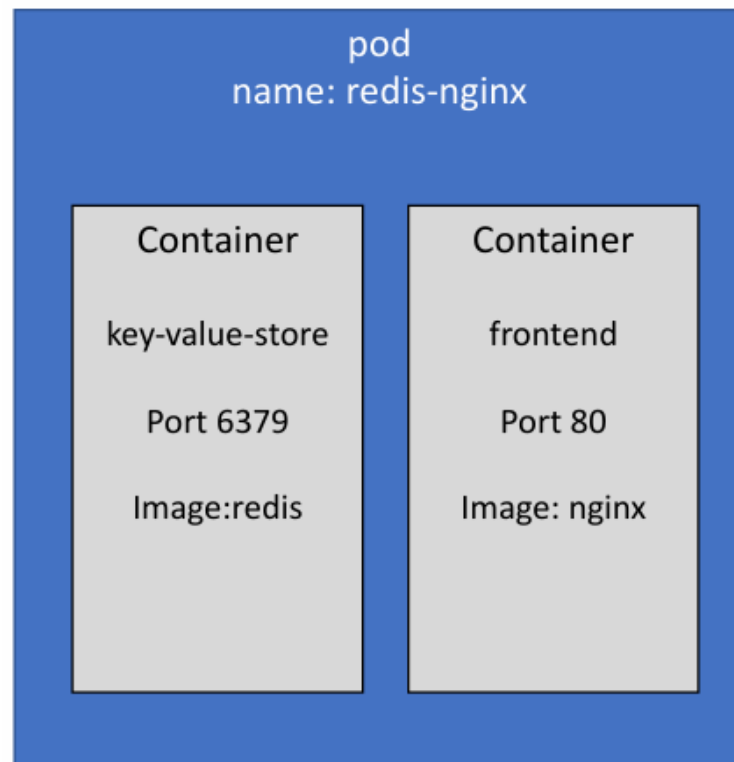
```
kubectl delete pod nginx-pod
kubectl get pods
```

3.1 K8s 두 번째 실습

■ 실습 순서

- 1. nginx-redis-pod.yaml 파일 생성
- 2. kubectl apply -f nginx-redis-pod.yaml
- 3. kubectl get pods
- 4. kubectl delete -f nginx-redis-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: redis-nginx
  labels:
    app: web
spec:
  containers:
    - name: key-value-store
      image: redis
      ports:
        - containerPort: 6379
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
```



3.1 Pod Lifecycle

- 다음은 Pod를 생성했을 때 클러스터 내부에서 동작되는 단계이다.
 - 1. YAML로 Pod spec 제출
 - `kubectl apply -f pod.yaml`
 - 2. API Server가 요청을 받고 etcd에 저장
 - API Server가 리소스를 검증/등록 (etcd에 영구 저장)
 - 3. Scheduler가 Pod를 감지하고 노드를 선택
 - 자원(CPU/RAM), 조건 등 고려하여 가장 적절한 노드에 매핑
 - 4. Kubelet이 노드에서 Pod 생성 수행
 - 해당 노드의 Kubelet이 API Server를 지켜보다가, 해당 노드에 매핑됨을 확인하고
 - 컨테이너 런타임(Containerd/Docker 등)에게 생성 지시
 - 5. 컨테이너 런타임이 컨테이너 실행
 - image pull, 컨테이너 생성/시작, 네트워크/볼륨 연결
 - 6. Pod 상태가 계속 etcd에 반영됨
 - Running / Pending / CrashLoopBackOff 등 상태 업데이트
 - kubectl로 조회하는 상태는 결국 etcd에 반영된 정보를 기반으로 함

3.2 Pod 리소스 관리

■ 개념

- 하나의 노드(Node)는 여러 Pod가 동시에 자원(CPU, RAM)을 공유한다.
- Pod에 대한 리소스 관리를 하지 않으면
 - 특정 Pod가 자원을 독점
 - 다른 Pod 장애 발생
 - 그로 인해 노드 전체가 불안정해지기 때문에 **Pod 단위 자원 관리**는 필수임
 - <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/#requests-and-limits>

■ 관리 방법 2가지

- K8s 는 다음 2 가지를 기준으로 스케줄링 (어디에 배치할지)과 실행 중 자원 사용을 제한한다.
- Pod 안의 모든 컨테이너에 개별 적용

개념	설명
Requests	Pod 가 최소한으로 필요로 하는 자원
Limits	Pod 가 최대로 사용할 수 있는 자원

3.2 Pod 리소스 관리

■ Requests

- Pod가 정상 동작하기 위해서 반드시 보장 받아야 하는 최소 자원
- 역할
 - 스케줄러 기준
 - 스케줄러는 Requests 기준으로 노드를 선택
 - 요청 자원을 수용할 수 없는 노드에는 절대 배치하지 않음
 - 자원 예약 효과
 - 해당 Pod가 실행되는 동안, 요청한 자원은 항상 확보됨

ex.

resources:

requests:

cpu: "500m"	# 0.5 Core
memory: "256Mi"	# 256 X 1024 X 1024 byte

3.2 Pod 리소스 관리

■ Limits

- Pod가 최대 사용할 수 있는 자원의 상한선
- 역할
 - 과도한 자원 사용 방지
 - 특정 Pod가 모든 노드 자원을 독점하지 못하도록 제한
 - 강제 제어
 - CPU 초과 시 속도 제한 (Throttle)
 - Memory 초과 시 컨테이너 강제 종료 (OOMKill)

ex.

resources:

requests:

cpu: "500m" # 0.5 Core

memory: "256Mi" # 256 X 1024 X 1024 byte

limits:

cpu: 1 # 1 Core

memory: "512Mi" # 512 X 1024 X 1024 byte

3.2 Pod 리소스 관리

■ Requests vs Limits

항목	Requests	Limits
목적	배치 기준	실행 제한
적용 시점	Pod 생성 시	Pod 실행 중
기준 주체	Scheduler	Kubelet
초과 시	배치 불가	CPU는 제한 / Memory는 종료

■ 단위

- CPU 단위
 - 1 CPU : 1 물리 코어 (Bare metal 기준)
 - m (밀리코어) : 1 CPU를 1,000 개로 쪼갠 단위
 - ex. 1000m : 1 CPU
 - 500m : 0.5 CPU
- Memory 단위
 - Ki (Kibibyte) : 1,024 Bytes
 - Mi (Mebibyte) : 1,024 Kib
 - Gi (Gibibyte) : 1,024 Mib

3.2 Pod 리소스 관리 실습

■ 실습 순서

- 1. `kubectl apply -f pod-resource-requests-limit.yaml`
- 2. `kubectl get pod stress-cpu -w` # 120초 동안 Running 상태 확인 가능
- 3. `kubectl top pod stress-cpu` # 사용중인 CPU 자원 확인 가능 (500m 근처), 수집 시간이 필요.
- 4. `kubectl describe pod stress-cpu` # 제한/요청 확인

```
apiVersion: v1
kind: Pod
metadata:
  name: stress-cpu
spec:
  restartPolicy: Never
  containers:
    - name: stress
      image: polinux/stress
#   args: ["stress", "--cpu", "1", "--timeout", "120s", "--verbose"]
      args: ["stress", "--vm", "1", "--vm-bytes", "300M", "--timeout", "120s", "--verbose"]
      resources:
        requests:
          cpu: "200m"
          memory: "64Mi"
        limits:
          cpu: "500m"
          memory: "128Mi"
```

3.3 Namespaces

■ 개념

- 하나의 물리적 K8s 클러스터를 여러 '가상 클러스터' 처럼 나눠 쓰게 해주는 논리적 구획임
- 필요한 이유
 - 클러스터 하나에 서비스/team/환경(dev, prod)등이 섞이면 문제가 심각해짐
 - ex.
 - 리소스 이름 충돌
 - 접근 제어 가능
 - 자원 남용 방지
 - 네트워크 통신 제한
 - Namespace는 이러한 문제들을 새로운 클러스터를 만들지 않고 해결 가능한 방법임
 - <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>

■ 핵심 특징

- 하나의 리소스는 하나의 Namespace에만 속한다.
- 리소스 이름은 Namespace 내부에서만 유일하면 된다.
- 중첩 불가
- default Namespace 가 제공된다.
- 모든 리소스가 Namespace를 사용하는 것은 아니다. (Node, Namespace, PersistentVolume 은 사용불가)

3.3 Namespaces

■ 주요 기능

- 네트워크 통신 허용/차단 (NetworkPolicy)
 - Namespace 단위로 통신 정책 설정 가능
- 권한 제어(RBAC) 제어
 - 특정 사용자/서비스 계정이 특정 namespace만 접근하도록 제한 가능
- 자원 할당 제한 (ResourceQuota/LimitRange)
 - Namespace별로 CPU/메모리/Pod 개수 등을 제한

■ 장단점

- 클러스터 하나로 멀티팀/멀티환경 운영 가능
- 이름 충돌 해결
- RBAC/Quota/NetworkPolicy 적용범위가 명확함
- 관리 단위(팀, 프로젝트, 환경)에 맞춰 운영하기 좋음

3.3 Namespaces

■ 주요 명령어

용도	명령어	설명
네임스페이스 목록 조회	kubectl get namespaces kubectl get ns	클러스터에 존재하는 모든 Namespace 조회
네임스페이스 상세 정보	kubectl describe ns <이름>	특정 Namespace 상태, 이벤트 확인
네임스페이스 생성	kubectl create ns <이름>	지정된 이름으로 Namespace 생성
네임스페이스 삭제	kubectl delete ns <이름>	Namespace 및 내부 리소스 전체삭제
특정 Namespace 자원 조회	kubectl get pod -n <이름> kubectl get service -n <이름> kubectl get svc -n <이름> kubectl get deploy -n <이름>	지정된 Namespace 기준으로 Pod 조회 지정된 Namespace 기준으로 service 조회 지정된 Namespace 기준으로 deployment 조회
특정 Namespace 자원 삭제	kubectl delete pod hello -n <이름>	지정된 Namespace 기준으로 Pod 삭제
모든 Namespace 대상 조회	kubectl get pod -A kubectl get svc -A kubectl get deploy -A kubectl get all -A	모든 Namespace의 Pod 조회 모든 Namespace의 Service 조회 모든 Namespace의 Deployment 조회 모든 Namespace의 전체 리소스 조회
YAML 배포시 지정	kubectl apply -f pod.yaml -n <이름>	

3.3 Namespaces 실습

■ 실습 순서

- 1. `kubectl create namespace dev`
- 2. `kubectl get ns`
- 3. `kubectl apply -f pod.yaml -n dev`
- 4. `kubectl get pod -n dev`
- 5. `kubectl describe pod nginx-pod -n dev`
- 6. `kubectl get pod` # default 네임스페이스에서 조회됨
- 7. `kubectl get pod -A` # 모든 namespace 대상 조회
- 8. `kubectl delete ns dev`

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

3.4 Labels 과 Selector

■ Labels

- 개념
 - K8s 오브젝트를 식별/분류/그룹화하기 위한 선언적(Declarative) 메타데이터임
 - K8s 객체에 설정하는 이름표(tag) 같은 기능
 - 사람이 보기 쉽고, 시스템이 필터링하기 쉽게 설계됨
 - <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>
- 목적
 - 단순한 설명용이 아니라 운영을 위한 핵심 메커니즘임
 - 오브젝트 분류
 - Service, Deployment, AutoScaling과 연결
 - 조직 구조를 시스템에 자연스럽게 반영

■ Labels 주요 특징

항목	설명
형식	key=value
값 개수	반드시 하나의 key는 하나의 value만 가짐 (리스트 불가)
중복 여부	같은 key는 하나의 객체에 한 번만 가능
개수 제한	0 개 이상
대소문자	구분함 (Case-sensitive)
key 허용문자	시작은 영문자, 특수문자(-, _ , .) 가능, 공백 불가

3.4 Labels 과 Selector

■ 샘플

- `kubectl get pods --show-labels`

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
    tier: backend
    env: prod
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

3.4 Labels 과 Selector

■ Selector

- 개념
 - Label 조건을 이용해서 객체 그룹을 동적으로 선택하는 방법임
- 특징
 - 특정 Label 조건과 매칭되는 객체 집합 생성
 - 새로운 객체가 생기면 자동 포함
 - 직접 연결이 아닌 조건 기반 연결임

■ Selector 종류

종류	사용 연산자	예시
Equality-based Selector	=, ==, !=	app=web, tier != backend # yaml selector: app: web
Set-based Selector	in , notin	env in (prod, stage) , tier notin (cache) # yaml selector: matchExpressions: - key: env operator: in values: - prod - stage

3.4 Labels 과 Selector

■ 주요 명령어

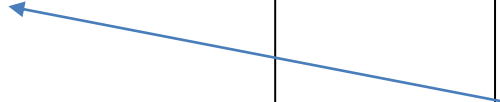
용도	명령어	설명
모든 pod + Label 조회	<code>kubectl get pods --show-labels</code>	가장 기본 명령어
특정 pod Label만 조회	<code>kubectl get pods web-pod --show-labels</code>	빠른 확인
Label 기준 조회	<code>kubectl get pods -l app=web</code> <code>kubectl get pods -l app!=web</code> <code>kubectl get pods -l 'env in (prod, stage)'</code> <code>kubectl get pods -l 'env notin (prod, stage)'</code>	동등비교 부등비교 IN 연산자 NOTIN 연산자
여러 Label 기준 조회	<code>kubectl get pods -l app=web, tier=backend</code>	AND 조건
Label 존재 여부	<code>kubectl get pods -l app</code>	key만 존재하면 매칭
Pod에 Label 추가	<code>kubectl label pod web-pod app=web</code>	기본 추가
여러 Label 추가	<code>kubectl label pod web-pod app=web tier=backend</code>	동시에 추가
기존 Label 값 변경	<code>kubectl label pod web-pod app=db --overwrite</code>	--overwrite 옵션 필수
Pod Label 삭제	<code>kubectl label pod web-pod app-</code>	- 지정하면 삭제
여러 Label 삭제	<code>kubectl label pod web-pod app- tier- env-</code>	동시에 삭제

3.4 Labels 과 Selector

■ 샘플

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod-1
labels:
  app: web
  tier: frontend
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
selector:
  app: web
  ports:
  - port: 80
    targetPort: 80
```



```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod-2
labels:
  app: web2
  tier: frontend
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

3.5 ReplicaSet

■ 개념

- 항상 지정한 개수(replica)의 Pod가 실행되도록 보장하는 컨트롤러임.
- Pod가 죽어도, 노드가 사라져도 개수만큼 다시 만듦.
- <https://kubernetes.io/ko/docs/concepts/workloads/controllers/replicaset/>

■ 목적

- Pod 개수 유지
- 서비스 가용성 보장
- 동일한 Pod를 여러 개 실행

■ 필요한 이유

- Pod만 사용할 경우
 - Pod는 1회성, Pod 장애발생 시 자동 복구 안됨
- ReplicaSet 사용할 경우
 - **Pod 장애발생 시, 즉시 새 Pod 생성**
 - 사람이 개입하지 않아도 **자동 복구**

3.5 ReplicaSet 구성요소 3가지

■ Replicas

- 유지할 Pod 개수
 - ex. **replicas: 3**
 - 항상 3개의 Pod 가 실행되도록 유지
 - Pod가 죽으면 새로운 Pod 생성
 - 수동/자동으로 scale 가능

■ Selector

- Pod 선택 기준
 - ex. **selector:**
 - matchLabels:**
 - app: web**
 - ReplicaSet이 관리할 Pod를 식별하는 기준
 - Label 기반으로 Pod를 선택

■ Pod Template

- Pod 설계도
 - ex. **template:**
 - metadata:**
 - labels:**
 - app: web**
 - spec:**
 - containers:**
 - **name: nginx**
 - image: nginx**
- 새로운 Pod를 만들 때 사용하는 설계도
- ReplicaSet은 Pod를 직접 수정하지 않고 template 기반으로 새로 생성됨
- **selector와 template의 label은 반드시 일치해야됨**

3.5 ReplicaSet

■ 샘플

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx
        ports:
        - containerPort: 80
```

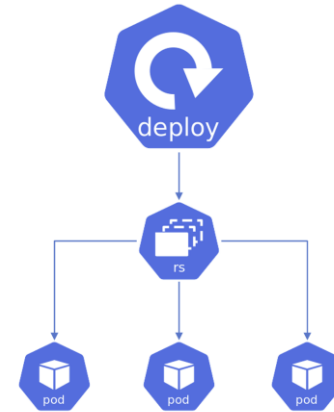
3.6 Deployment

■ 기본 개념

- Pod를 직접 관리하지 않고, ReplicaSet을 통해서 Pod를 안정적으로 관리/배포하는 상위 컨트롤러임.
- <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

■ 필요한 이유

- Pod만 사용할 경우
 - Pod는 1회성, Pod 장애발생 시 자동 복구 안됨
 - pod 개수 늘리려면 수동 작업
 - 새 버전 배포 시 서비스 중단 위험
 - 이전 버전으로 되돌리기 어려움
- **Deployment는 이 모든 문제를 해결 가능**



3.6 Deployment

■ 주요 핵심 개념

- Deployment는 ReplicaSet을 관리하는 고수준 컨트롤러이다.
 - ReplicaSet을 생성/관리하기 때문에 사용자는 직접 ReplicaSet을 다루지 않음.
- Deployment에는 desired state 를 선언한다.
- Deployment는 actual state를 desired state로 변경할 때, 점진적(Controlled Rate)으로 변경한다.
 - Pod를 한 번에 모두 바꾸지 않고 점진적으로 변경함.
 - **무중단 배포(Rolling Update)의 핵심**
- Deployment는 새 버전 배포 전략을 제어(Rollout)하거나 이전 버전으로 즉시 복구(Rollback)가 가능하다.
 - Rollout - 새 배포 버전 전략 제어
 - Rollback - 이전 버전으로 즉시 복구
 - History - 배포 이력 관리
 - Pause/Resume - 배포 일시중지/재개

■ 주요 기능 정리

- 자동 복구 (Self-Healing)
- 스케일링 (Scaling)
- 롤링 업데이트 (Rolling Update)
- 롤백(Rollback)

3.6 Deployment

■ 주요 명령어

용도	명령어	설명
Deployment 목록	kubectl get deployment kubectl get deploy	클러스터 내 Deployment 목록
Deployment 상세	kubectl describe deploy web	이벤트, ReplicaSet, 상태 확인
Deployment 생성	kubectl apply -f deployment.yaml kubectl create deployment web --image=nginx	선언적 생성 (권장)
Deployment 삭제	kubectl delete deployment web	관련 ReplicaSet, Pod 모두 삭제
scale	kubectl scale deploy web --replicas=5	즉시 확장/축소
배포 상태 확인	kubectl rollout status deploy web	진행 중/완료 확인
배포 이력 조회	kubectl rollout history deploy web	이전 버전 목록
특정 리비전 확인	kubectl rollout history deploy web --revision=2	상세 변경 내역
롤백	kubectl rollout undo deploy web	이전 버전으로 복구
특정 버전 롤백	kubectl rollout undo deploy web --to-revision=2	원하는 버전으로 복구
이미지 변경	kubectl set image deploy web web=nginx:1.26	롤링 업데이트 수행
배포 일시 중지	kubectl rollout pause deploy web	변경 사항 적용 중단
배포 재개	kubectl rollout resume deploy web	누적 변경사항 반영

3.6 Deployment Strategy

■ 개념

- Deployment Strategy는 어플리케이션을 변경하거나 업그레이드하는 방법이다.
- 새 버전 배포 시 기존 Pod를 언제 / 어떻게 / 어떤 순서로 교체할지 결정하는 규칙으로 Deployment에 의해 제어된다.

■ 종류

▪ Rolling Update (기본)

- 기존 버전의 인스턴스를 하나씩 새로운 버전으로 교체하여 모든 인스턴스가 새 버전이 될 때까지 점진적으로 배포하는 방식이다.
- 서비스 중단이 없는 무중단 배포가 가능하여 안정성이 매우 높다.
- ex. strategy:

type: RollingUpdate

▪ Recreate

- 기존 버전의 모든 인스턴스를 종료한 뒤, 새로운 버전의 어플리케이션을 배포하는 방식이다.
- 서비스 중단이 발생되지만 배포 속도가 빠르다.
- ex. strategy:

type: Recreate

3.6 Deployment Strategy

■ RollingUpdate vs Recreate

항목	RollingUpdate	Recreate
기본 전략	○	X
무중단 배포	○	X
다운타임	X	○
배포 안정성	높음	낮음
배포 속도	느림	빠름
실무 사용 빈도	매우 높음	제한적

3.6 Deployment

■ 샘플

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3           # 원하는 Pod 개수
  strategy:
    type: RollingUpdate  # Deployment Strategy
  selector:
    matchLabels:
      app: web
  template:              # Pod 템플릿
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.25
          ports:
            - containerPort: 80
```

3.6 Deployment

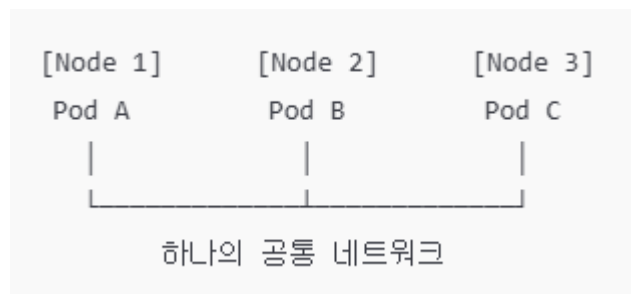
■ ReplicaSet vs Deployment

기능	ReplicaSet	Deployment
Pod 개수 유지	O	O
Pod 자동 복구	O	O
버전 관리	X	O
롤링 업데이트	X	O
롤백	X	O
배포 이력 관리	X	O
Pause/Resume	X	O
실무에서 직접 사용	X	O

3.7 K8s Network

■ 정의

- 클러스터내의 모든 Pod를 하나의 공통 네트워크에 있는 서버처럼 동작하게 만드는 네트워크 구조이다.
- 즉 클러스터 전체를 하나의 논리적 네트워크로 간주한다.



■ 핵심 개념

- 모든 Pod는 고유한 IP를 가진다.
- Pod 내부 컨테이너는 네트워크를 공유한다.
- Pod 간 통신은 직접 이루어진다.
- Docker 같은 컨테이너 포트 매핑은 거의 필요 없다.

3.7 K8s Network

■ Pod 중심 네트워크 구조

- 네트워크의 최소 단위는 컨테이너가 아니라 Pod 이다
- Pod마다 IP 1개 제공
- Pod 안의 컨테이너들은
 - 같은 IP 사용
 - 같은 IP를 사용하지만 Port는 달라야 됨
 - localhost로 통신

■ Pod와 Pod 통신 방식

- Pod는 어느 노드에 있든 다른 Pod와 직접 IP 통신 가능
- 클러스터 전체가 하나의 평면 네트워크(flat network)처럼 동작

```
Pod A (Node1) → Pod B (Node2)  
10.244.1.10 → 10.244.2.15
```

3.7 K8s Network

■ Docker vs K8s

구분	Docker	K8s
네트워크 기준	컨테이너	Pod
IP 할당	컨테이너별	Pod별
내부 통신	컨테이너 IP	localhost
Pod/컨테이너 간 통신	제한적	기본 허용
포트 매핑	필수 ex. -p 8080:80	거의 불필요
외부 노출 방식	Host 포트	Service
네트워크 철학	단일 서버	분산 시스템

3.8 Service

■ 개념

- Service는 Pod 집합에 대한 '고정된 네트워크 접속 창구' 역할을 제공하는 K8s 리소스이다.
- 즉 Service를 통해서 Pod에 접속할 수 있다.

■ Service가 필요한 이유

- Pod는 임시 객체이기 때문에 재생성될 때마다 매번 IP가 바뀜.
 - Pod는 desired state를 맞추기 위해서 계속 생성/삭제/재생성 됨.
- 따라서 Pod IP에 직접 의존하면 통신이 원활하지 못하게 됨.
- 하지만 Service는 Pod가 바뀌어도 변하지 않는 주소(DNS + 가상 IP)를 제공하기 때문에 원활한 통신이 가능함.

■ 핵심 역할

역할	설명
고정 진입점 제공	Pod IP가 바뀌어도 Service 주소는 고정
Pod 자동 추적	selector(Label) 기반으로 대상 Pod 자동 선택
로드 밸런싱	L4 (TCP/UDP) 기반 분산 (IP:Port 기준)
포트 매핑	Service Port를 Pod TargetPort로 연결 Service Port: 외부/클러스터에서 보이는 port TargetPort: 실제 Pod 컨테이너가 리스닝하는 port
표준 통신 방식	클러스터 내부에서 Pod를 노출하는 가장 선호되는 방식

3.8 Service

■ 장점

장점	설명
Pod 변경에 안전	IP 변경에 영향 없음
설정 단순	selector만 잘 사용하면 자동으로 연결
자동 로드밸런싱	별도 설정 불필요
표준 패턴	K8s 내부 통신의 기본

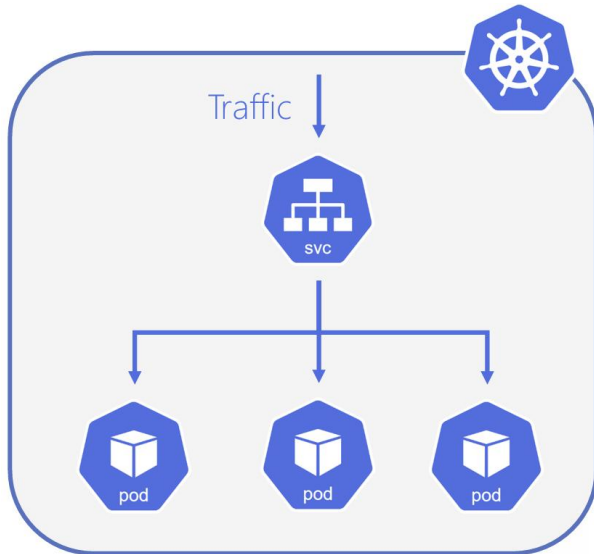
■ 단점

단점	설명
L7 라우팅 불가	URL, Host 기반 분기 불가
외부 노출 제한	ClusterIP는 외부 접근 불가
selector 의존	label 실수 시 트래픽 불가

3.8 Service 타입 종류 1

■ ClusterIP (기본값)

- Service를 클러스터 내부 IP로만 노출함
 - 즉, 외부 PC / 브라우저에서 직접 접근 불가
 - 따라서 외부 노출 없는 내부 전용 서비스에서 사용됨
- Service 이름 자체가 DNS 주소로 처리함
 - K8s는 내부에 DNS 서버(CoreDNS)를 운영
 - 같은 Namespace 에서 접근
 - » **http://svc-name** ex. http://backend
 - 다른 Namespace 에서 접근
 - » http://svc-name.namespace ex. http://backend.default / http://backend.default.svc.cluster.local



```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
  type:
    ClusterIP
```

3.8 Service 타입 종류 1

■ ClusterIP 샘플

```
# service-web-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

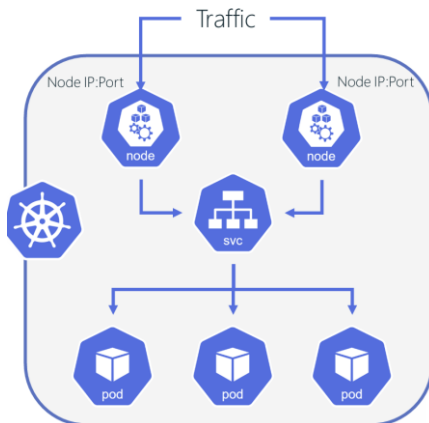
```
# service-web-service-clusterip.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: web          # Pod label과 반드시 일치
  ports:
    - port: 80        # Service 포트
      targetPort: 80   # Pod 포트
  type: ClusterIP     # 기본값 (생략 가능)
```

- `kubectl apply -f .\service-web-pod.yaml`
- `kubectl get pod`
- `kubectl apply -f .\service-web-service-clusterip.yaml`
- `kubectl get service`
- `kubectl exec -it nginx-pod -- sh`
 - # `curl http://localhost`
 - # `curl http://nginx-service` # 서비스명 직접 호출
 - # `curl http://nginx-service.default`

3.8 Service 타입 종류 2

■ NodePort

- Service를 각 노드(Node)의 특정 포트(NodePort)로 외부에 노출한다.
 - 외부 클라이언트는 **노드IP:NodePort** 로 접속하고
 - 그 트래픽이 K8s의 Service를 통해서 Pod로 전달된다.
- NodePort Service를 만들면 내부적으로 다음과 같이 2개가 생성된다.
 - ClusterIP (내부 통신용 IP)
 - NodePort (외부에서 들어오는 포트)
- 접근 방식
 - 외부 접근
 - **http:// <NodeIP>:<NodePort>**
 - 내부 접근
 - http://svc-name / http://svc-name.default



```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  type:
    NodePort
```

3.8 Service 타입 종류 2

■ NodePort 샘플

```
# service-web-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

```
# service-web-service-nodeport.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  selector:
    app: web          # Pod label과 반드시 일치
  ports:
    - port: 80        # Service 포트
      targetPort: 80  # Pod 포트
      nodePort: 30080  # 외부에서 접근할 포트 (선택: 직접 지정)
  type: NodePort
```

3.8 Service 타입 종류 3

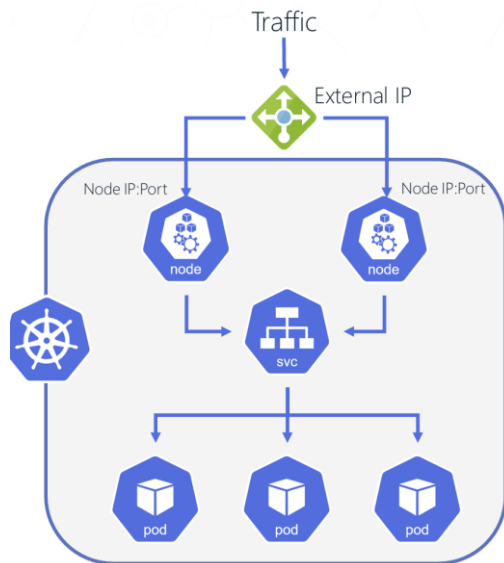
■ LoadBalancer

- 클라우드 제공자의 로드밸런서를 자동으로 생성하여 외부에서 고정 IP(또는 DNS)로 Service에 접근할 수 있도록 도와주는 Service 타입이다.
- LoadBalancer Service를 만들면 내부적으로 다음과 같이 3개가 생성된다.
 - ClusterIP (내부 통신용 IP)
 - NodePort (외부에서 들어오는 포트)
 - 외부 LoadBalancer (External IP)
- 접근 방식
 - 외부 접근
 - `http:// <External-IP>`
 - 내부 접근
 - `http://svc-name / http://svc-name.default`
- 주요 핵심
 - 외부 사용자는 노드 IP를 몰라도 됨
 - 로드밸런서는 노드 단위 분산
 - Service는 Pod 단위 분산

3.8 Service 타입 종류 3

■ **LoadBalancer**

- 사용하는 경우
 - 클라우드 환경 (AWS / GCP / Azure)
 - 외부 사용자에게 바로 서비스 공개
 - HTTPS, 고정 IP, 헬스체크 필요



```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
  type:
    LoadBalancer
```

3.8 Service 타입 종류 3

■ LoadBalancer 샘플

```
# service-web-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

```
# service-web-service-lb.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  selector:
    app: web          # Pod label과 반드시 일치
  ports:
    - port: 80        # Service 포트
      targetPort: 80   # Pod 포트
  type: LoadBalancer
```


3.9 Ingress

■ 개념

- 클러스터 외부(인터넷/사내망)에서 들어오는 HTTP/HTTPS 트래픽을 클러스터 내부의 Service(ClusterIP)로 라우팅해주는 규칙(Rule) 모음 리소스이다.

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

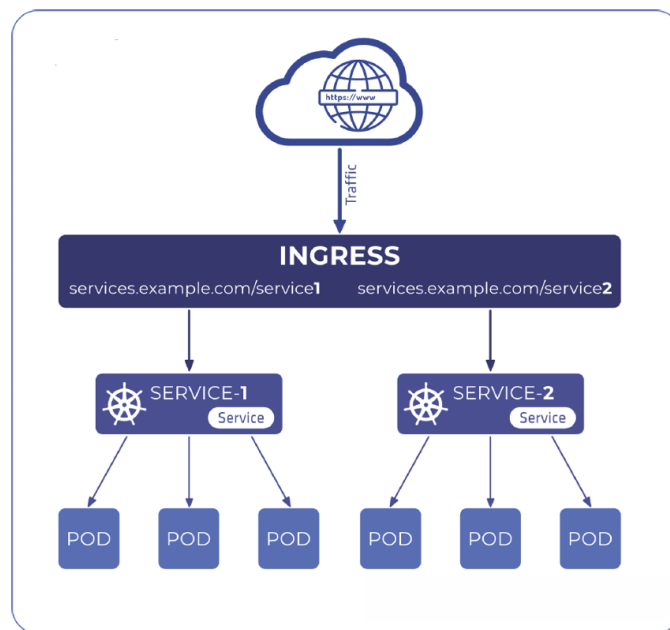
ex.

example.com/ 요청하면 web-service 가 처리

example.com/api 요청하면 api-service 가 처리

■ 각 Service 타입 방식의 장단점

- NodePort 방식
 - 장점: 단순함
 - 단점:
 - 서비스마다 포트가 늘어남 (ex. 30080, 30081 ...)
 - 도메인 기반 라우팅/경로 기반 라우팅 지원 안됨
 - 운영/보안(방화벽 규칙 등) 관리가 번거로우
- LoadBalancer 방식
 - 장점: 외부 LB가 제공되어 편리함 (클라우드)
 - 단점:
 - 서비스마다 LoadBalancer가 추가되면 비용/관리 증가



3.9 Ingress

■ Ingress 방식의 장단점

- 장점:
 - 단일 진입점(하나의 LB/하나의 IP)에서 여러 서비스를 호스트/경로 기준으로 분기 가능
 - TLS(HTTPS) 종료를 Ingress에서 처리 가능
 - URL 구조로 서비스 분리 가능 (ex. /api, /web, /admin 등)
- 단점:
 - Ingress Controller 설치/운영이 필요

■ Ingress 필수 구성요소 2 가지

- Ingress Resource (YAML)
 - 어떤 요청이 오면 어떤 Service로 보낼지 규칙 정의 (라우팅 정보)
ex. example.com/ 요청하면 web-service 가 처리

<https://kubernetes.io/docs/concepts/services-networking/ingress/#the-ingress-resource> 가이드

- Ingress Controller
 - Ingress Resource(YAML)를 읽어서 실제 라우팅을 수행하는 컨트롤러
 - 대표적으로 **NGINX Ingress Controller**, Traefik, HAProxy, Kong 등
 - <https://kubernetes.github.io/ingress-nginx/deploy/> 설치 가이드

3.9 Ingress

■ Ingress Controller

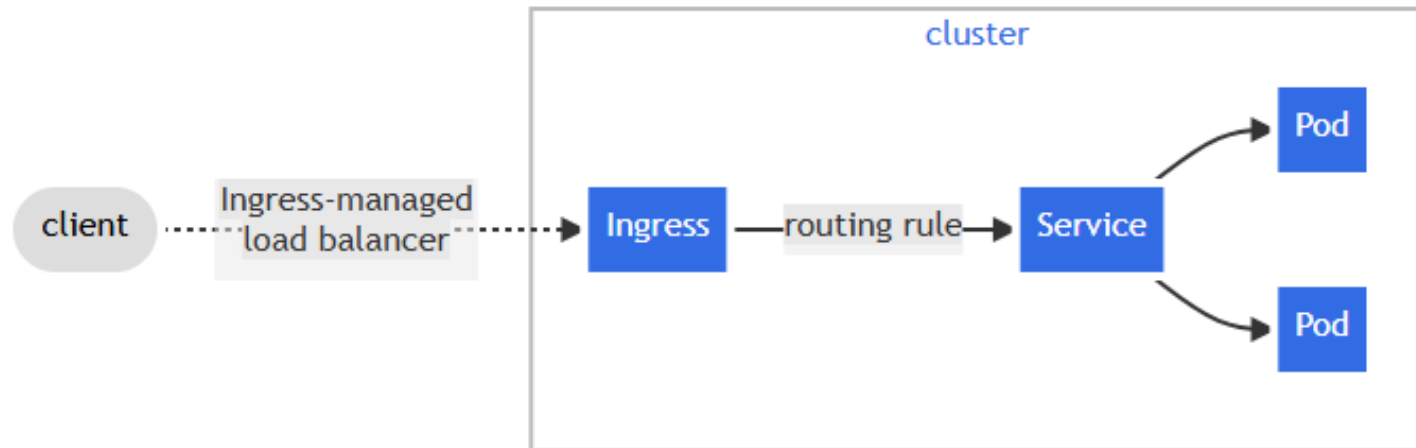
- Ingress는 외부 요청을 어디(Service)로 보낼지에 대한 규칙만 적어 놓은 설정 정보일 뿐이다.
- Ingress 자체는 트래픽을 처리하거나 라우팅을 수행하지 않는다.
- 실제로 요청을 받아서 전달하는 역할은 Ingress Controller가 담당한다.

■ 특징

- K8s는 Ingress라는 리소스 타입만 제공할 뿐, 실제로 동작하는 Ingress Controller는 기본 제공하지 않는다.
- 따라서 클러스터 관리자가 Ingress Controller를 직접 설치해야 한다.
- <https://kubernetes.github.io/ingress-nginx/deploy/> 설치 가이드 참조

3.9 Ingress

■ Ingress 아키텍처



3.9 Ingress

■ Ingress 샘플

```
# ingress-deployment-service.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

3.9 Ingress

■ Ingress 샘플

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: hashicorp/http-echo:1.0
          args:
            - "-text=hello from api"
          ports:
            - containerPort: 5678
---
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: api-svc
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 5678
```

3.9 Ingress

■ Ingress 샘플

```
# ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ing
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-svc
                port:
                  number: 80
```

4. Kubernetes Config

4.1 template env

■ 개념

- Pod Template 안에서 컨테이너의 환경 변수 (Environment Variables)를 정의하는 곳
- <https://kubernetes.io/docs/tasks/inject-data-application/define-environment-variable-container/>

■ 샘플

```
apiVersion: v1
kind: Pod
metadata:
  name: envar-demo
  labels:
    purpose: demonstrate-envvars
spec:
  containers:
    - name: envar-demo-container
      image: gcr.io/google-samples/hello-app:2.0
      env:
        - name: GREETING
          value: "Hello from the environment"
        - name: FAREWELL
          value: "Such a sweet sorrow"
        - name: MESSAGE
          value: "$(GREETING) $(FAREWELL)"
```

4.1 template env

■ 실습

```
# config-env.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: nginx
      env:
        - name: LOG_LEVEL
          value: "debug"
        - name: APP_MODE
          value: "dev"
```

4.2 ConfigMaps

■ 개념

- 어플리케이션의 설정 데이터를 컨테이너 이미지 밖으로 분리해서 K8s 오브젝트로 관리하게 해주는 리소스이다.
- <https://kubernetes.io/docs/concepts/configuration/configmap/>

■ 특징

- 기밀 정보가 아닌 데이터를 key-value 형태로 저장하기 위해 사용한다.
- 컨테이너 이미지에서 환경별 설정(environment-specific configuration)을 분리할 수 있어, 어플리케이션을 다양한 환경에서 쉽게 이식(portable)할 수 있다.
- Pod는 ConfigMap을 다음과 같은 방식으로 사용할 수 있다.
 - 환경 변수(Environment Variable)로 사용
 - ConfigMap 수정 시 자동 업데이트가 되지 않으며 변경 사항을 적용하려면 Pod 재시작이 반드시 필요하다.
 - **kubectl rollout restart deploy <deployment-name>**
 - Volume에 설정 파일 형태로 마운트하여 사용
 - ConfigMap이 업데이트되면 자동으로 반영된다.

4.2 ConfigMaps

■ 실습1- Environment

```
# config-ConfigMaps-Environment.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-env
data:
  APP_MODE "dev"
  LOG_LEVEL: "debug"
```

```
# config-ConfigMaps-Environment-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: nginx
          env:
            - name: LOG_LEVEL
              valueFrom:
                configMapKeyRef:
                  name: cm-env
                  key: LOG_LEVEL
            - name: APP_MODE
              valueFrom:
                configMapKeyRef:
                  name: cm-env
                  key: APP_MODE
```

5.2 ConfigMaps

■ 실습2- Volume

```
# config-ConfigMaps-Environment.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cm-env
data:
  APP_MODE "dev"
  LOG_LEVEL: "debug"
```

```
# config-ConfigMaps-Volume-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      volumes:
        - name: config-volume
          configMap:
            name: cm-env

      containers:
        - name: app
          image: nginx
          volumeMounts:
            - name: config-volume
              mountPath: /etc/config
```

4.3 Secrets

■ 개념

- K8s에서 비밀번호, API Key, 인증서, 토큰 같은 민감한 데이터를 저장하기 위한 리소스이다.
- ConfigMap과 거의 동일한 구조지만 인코딩 데이터 전용이라는 점이 다르다.
- <https://kubernetes.io/docs/concepts/configuration/secret/>

■ 일반 환경 변수 및 ConfigMap 문제점

- YAML에 비밀번호 노출 가능
- Git 저장소에 push 위험 가능
- Pod 로그로 유출 가능

■ 특징

- 민감 정보 저장
- Base64 인코딩 저장
- 필요할 때만 Pod에 주입 (env/volume)

4.3 Secrets

■ 실습1- Environment

```
# config-Secrets-Environment.yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-env
type: Opaque
data:
EMAIL: c2VjcmV0QGV4YW1wbGUuY29t # secret@example.com

stringData:
  USERNAME: admin
  PASSWORD: P@ssw0rd!
```

4.3 Secrets

■ 실습1- Environment

```
# config-Secrets-Environment-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: secret-config-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: nginx
          env:
            - name: EMAIL
              valueFrom:
                secretKeyRef:
                  name: secret-env
                  key: EMAIL
```

```
- name: USERNAME
  valueFrom:
    secretKeyRef:
      name: secret-env
      key: USERNAME
- name: PASSWORD
  valueFrom:
    secretKeyRef:
      name: secret-env
      key: PASSWORD
```


5. Kubernetes Storage

5.1 K8s Volume

■ 개념

- Pod 안의 컨테이너들이 사용할 수 있는 '공유/지속 저장 공간'을 의미한다.
- <https://kubernetes.io/docs/concepts/storage/volumes/>

■ K8s Volume이 필요한 이유

- 컨테이너 내부 디스크에 저장된 파일은 임시적(ephemeral)이다.
 - 컨테이너는 일회용 프로세스와 가깝기 때문에 내부 파일 시스템은 컨테이너 생명 주기에 종속적임
- 문제1
 - 컨테이너가 제거(crash)되면 파일이 사라진다.
 - kubelet은 컨테이너를 다시 시작하면, 완전히 초기화된 상태(clean state)로 시작한다.
 - 따라서 로그파일/업로드 파일/임시 데이터 등 데이터 유지가 불가능
- 문제2
 - 같은 Pod 안에서 실행 중인 컨테이너들 사이에서 파일을 공유해야 할 때 문제가 발생된다.
 - Pod에는 여러 컨테이너들이 존재할 수 있고, 각 컨테이너마다 파일 시스템이 기본적으로 분리
 - 따라서 파일 공유가 불가
 - ex.
 - app 컨테이너 : 로그 파일 생성
 - sidecar 컨테이너: 로그 수집

5.1 K8s Volume

■ Volume 특징

- **Volume은 컨테이너가 아니라 Pod에 속한다.**
 - K8s 에서 Pod는 배포 및 스케줄링의 최소 단위
 - Pod내의 컨테이너들은 같은 네트워크(IP)와 같은 Volume을 공유
- **Volume은 컨테이너 파일 시스템과 분리된다.**
 - 컨테이너는 기본적으로 자신만의 root filesystem(rootfs)를 가짐
 - 따라서 컨테이너가 재시작 시 초기 상태로 설정
 - Volume은 rootfs와 물리적으로 분리되어 외부 저장소처럼 취급
 - 따라서 컨테이너 재시작과 무관하게 유지 가능
- **지속성(persistence)은 Volume 타입에 따라 달라진다.**
 - emptyDir : Pod가 살아있는 동안만 유지 (Pod 삭제시 같이 삭제됨)
 - hostPath : 노드 로컬 디스크에 저장 (노드 바뀌면 사용 불가)
 - PV/PVC : 클러스터 저장소로 영속 저장 (운영에서 가장 일반적)

■ 생명주기에 따른 분류

구분	설명
Ephemeral Volume	Pod의 생명주기 동안만 존재
Persistent Volume	Pod가 사라져도 계속 존재

5.2 Ephemeral Volume

■ 개념

- 어떤 어플리케이션은 추가 저장공간이 필요하지만 그 데이터가 재시작 이후에도 유지될 필요가 없는 경우에 사용될 수 있는 임시 볼륨이다.
- ex.
 - 캐시 데이터
 - 성능 향상을 위한 보조 데이터 역할
 - 읽기 전용 입력 데이터
 - 설정 데이터 (Configuration)
 - 비밀 키(Secret keys)

■ 주요 특징

- 캐시 / 임시 작업 디렉터리 / 설정파일 / 인증 키 같은 데이터는 필요하지만 영구적일 필요가 없다.
- Pod의 생명주기를 따른다.
 - 컨테이너가 재시작하면 Volume 유지됨
 - Pod 삭제하면 Volume 삭제됨

5.2 Ephemeral Volume

■ 종류

▪ EmptyDir

- 하나의 Pod 안에 있는 모든 컨테이너가 읽고 쓸 수 있는 임시 디렉터리
- Pod가 생성될 때 빈 디렉터리가 만들어지고 여러 컨테이너가 읽기/쓰기 및 공유 가능
- Pod가 삭제되면 EmptyDir도 같이 삭제됨
- ex.
 - 컨테이너 간 파일 공유
 - 캐시 데이터
 - 임시 디렉터리

▪ HostPath

- 호스트 노드의 실제 파일 또는 디렉터리를 Pod안으로 마운트하는 방식
- 다중 노드 클러스터 환경에서는 실용적이지 못함
- 컨테이너가 노드의 실제 디스크 경로를 그대로 사용
- 단, Pod가 다른 노드로 이동하면 파일 사용 불가
- Node 의존성 및 보안 위험 증가
- type 속성을 이용해서 제어 가능
 - **DirectoryOrCreate**: 경로가 없으면 빈 디렉터리 자동 생성
 - **Directory**: 해당 경로에 반드시 디렉터리가 존재해야 됨
 - **FileOrCreate**: 파일이 없으면 빈 파일 자동 생성
 - **File**: 해당 파일이 반드시 존재해야 됨

5.2 Ephemeral Volume

■ EmptyDir vs HostPath

구분	emptyDir	hostPath
저장 위치	Pod 전용 임시 공간	노드 파일 시스템
컨테이너 공유	가능	가능
Pod 삭제 시	데이터 같이 삭제	데이터 유지 가능 (단, 같은 노드에 재생성된 경우)
노드 의존성	없음	있음
멀티 노드 환경	안전	부적합
운영 권장	O	X

5.2 Ephemeral Volume

■ emptyDir 샘플

```
# ephemeral_volume_emptyDir.yaml
apiVersion: v1
kind: Pod
metadata:
  name: vol-emptydir-demo
spec:
  volumes:
    - name: shared-data
      emptyDir: {}

  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "while true; do date >> /shared/out.txt; sleep 2; done"]
      volumeMounts:
        - name: shared-data
          mountPath: /shared

    - name: reader
      image: busybox
      command: ["sh", "-c", "tail -f /shared/out.txt"]
      volumeMounts:
        - name: shared-data
          mountPath: /shared
```

5.2 Ephemeral Volume

■ hostPath 샘플

```
# ephemeral_volume_hostPath.yaml
apiVersion: v1
kind: Pod
metadata:
  name: vol-hostpath-demo
spec:
  volumes:
    - name: host-logs
      hostPath:
        path: /tmp/k8s-hostpath # 노드 파일 시스템
        type: DirectoryOrCreate

  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "echo hello >> /host/log.txt; sleep 3600"]
      volumeMounts:
        - name: host-logs
          mountPath: /host # 컨테이너 파일 시스템
```


5.2 Ephemeral Volume

■ emptyDir + hostPath 샘플

```
# ephemeral_volume_emptyDir_hostPath.yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-lab-pod
spec:
  volumes:
    - name: temp-vol
      emptyDir: { }

    - name: host-folder
      hostPath:
        path: /test.txt
        type: FileOrCreate
```

```
containers:
  - name: redis
    image: redis
    ports:
      - containerPort: 80
    volumeMounts:
      - name: temp-vol
        mountPath: /share

  - name: nginx
    image: nginx:1.17
    ports:
      - containerPort: 6379
    volumeMounts:
      - name: temp-vol
        mountPath: /usr/share/nginx/html
      - name: host-folder
        mountPath: /share/somefile
```

5.3 Persistent Volume

■ 개념

- Pod의 생명주기와 관계없이 존재하는 영구적인 스토리지이다.
 - 즉, Pod가 삭제되거나 재생성되어도 데이터는 유지된다.
- 스토리지의 내부 구조는 몰라도 되고, 필요한 만큼 요청해서 사용만 하는 구조이다.
- <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

■ 주요 요소

- PV (PersistentVolume)
 - 클러스터 안에 존재하는 하나의 저장 공간이다.
 - 즉, Namespace가 없는 클러스터 전역 리소스임
 - 다음 2 가지 방식으로 생성할 수 있다.
 - Static Provisioning : 관리자가 직접 미리 생성
 - Dynamic Provisioning : StorageClass를 이용한 동적 생성
- PVC (PersistentVolumeClaim)
 - 사용자가 스토리지를 요청하는 방식이다.
 - Pod가 CPU, Memory 같은 노드 리소스를 사용하는 것처럼 PVC는 PV라는 스토리지 리소스를 사용한다.
 - PV가 PVC에서 요청한 모든 조건을 만족해야 리소스를 사용할 수 있다. (binding)
 - ex.
 - 용량(Storage size): storage 1Gi 등
 - accessMode : ReadWriteOnce, ReadOnlyMany, ReadWriteMany
 - storageClass 등

5.3 Persistent Volume

■ PV 장점

- Pod가 재생성되어도 데이터 유지 가능
- 스토리지와 어플리케이션 분리 (배포/롤백/스케일링이 쉬움)
- 사용자(개발자)는 PVC로만 요청하고, 인프라 담당자는 PV/StorageClass로 제공 가능

■ PV 단점

- 스토리지 타입 별로 제약 존재
 - ex. 많은 블록 스토리지는 RWO(ReadWriteOnce) 중심
- 동적 프로비저닝/스토리지 드라이버(CSI) 구성에 따라 운영 난이도 증가

■ 접근 모드 (Access Mode)

- RWO (ReadWriteOnce)
 - 한 노드에 붙은 Pod들만 읽기/쓰기 가능 (일반적)
- ROX (ReadOnlyMany)
 - 여러 곳에서 읽기만 가능
- RWX (ReadWriteMany)
 - 여러 곳에서 동시에 읽기/쓰기 가능

5.3 Persistent Volume

■ Persistent Volume Type

- PV(Persistent Volume) 타입은 플러그인(plugin) 형태로 구현된다.
- 즉, K8s는 다양한 외부 스토리지 시스템을 플러그인 방식으로 연결하여 사용할 수 있다.
- 다음은 대표적인 Persistent Volume 플러그인 이다.

플러그인	설명
CSI (Container Storage Interface)	K8s에서 표준화된 스토리지 인터페이스 다양한 스토리지 벤더가 CSI 드라이버를 제공 ex. CephFS CSI, AWS EBS CSI, GCP PD CSI 등
Cephfs	Ceph 파일 시스템 기반 스토리지 분산 스토리지 환경에서 많이 사용됨 여러 Pod에서 동시에 접근 가능
fc (Fibre Channel)	고성능 SAN(Storage Area Network)환경에서 사용 기업용 스토리지에서 많이 사용되는 방식
nfs (Network File System)	네트워크 파일 시스템 기반 스토리지 여러 Pod에서 동시에 공유 가능 교육용/실습용으로 많이 사용됨
issi (SCSI over IP)	네트워크를 통해 원격 블록 스토리지 사용 IP 기반으로 SCSI 장치를 연결

5.3 Persistent Volume

■ Reclaim Policy (반환 정책)

- PV가 PVC와 연결이 끊어졌을 때, PV를 어떻게 처리할지를 결정하는 정책이다.
- 방법은 다음과 같다.
 - Retain
 - 데이터/볼륨을 유지
 - 다른 Pod에서 재사용 가능
 - 관리자가 수동으로 정리 필요
 - 운영에서 주로 사용
 - Delete (기본값)
 - PVC 삭제 시 실제 스토리지도 삭제
 - 클라우드 동적 PVC에서 주로 사용
 - 즉, PVC 삭제되면 PV 삭제되고 실제 스토리지까지 삭제됨

5.3 Persistent Volume

■ 실습1 - Static Provisioning

```
# persistent-volume-pv.yaml
apiVersion: v1
kind: PersistentVolume # Static Provisioning
metadata:
  name: pv-hostpath-1g
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce # hostPath 타입 활용 시 ReadWriteOnce만 가능
  persistentVolumeReclaimPolicy: Retain # PVC 삭제 시 PV는 유지
  storageClassName: manual # PVC의 storageClassName과 같으면 볼륨이 연결됨
  hostPath:
    path: /mnt/data/pv1 # K8s 내부 공간에서 /mnt/data/pv1 의 경로를 볼륨으로 설정
```

```
# persistent-volume-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-hostpath-1g
spec:
  accessModes:
    - ReadWriteOnce # 볼륨에 접근할 때의 권한
  storageClassName: manual # PV의 storageClassName과 같으면 볼륨이 연결됨
  resources:
    # PVC가 PV에 요청하는 리소스 정의
    requests:
      # 필요한 최소 리소스
      storage: 1Gi # PV가 최소 1Gi 이상은 되어야 됨
```

5.3 Persistent Volume

■ 실습1 - Static Provisioning

```
# persistent-volume-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pv-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pv-nginx
  template:
    metadata:
      labels:
        app: pv-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          volumeMounts:
            - name: data-volume
              mountPath: /usr/share/nginx/html # nginx 웹 루트
      volumes:
        - name: data-volume
          persistentVolumeClaim:
            claimName: pvc-hostpath-1g # 기존 PVC 이름
```

6. Autoscaling / Probes / Init Container

6.1 HPA (Horizontal Pod Autoscaler)

■ 개념

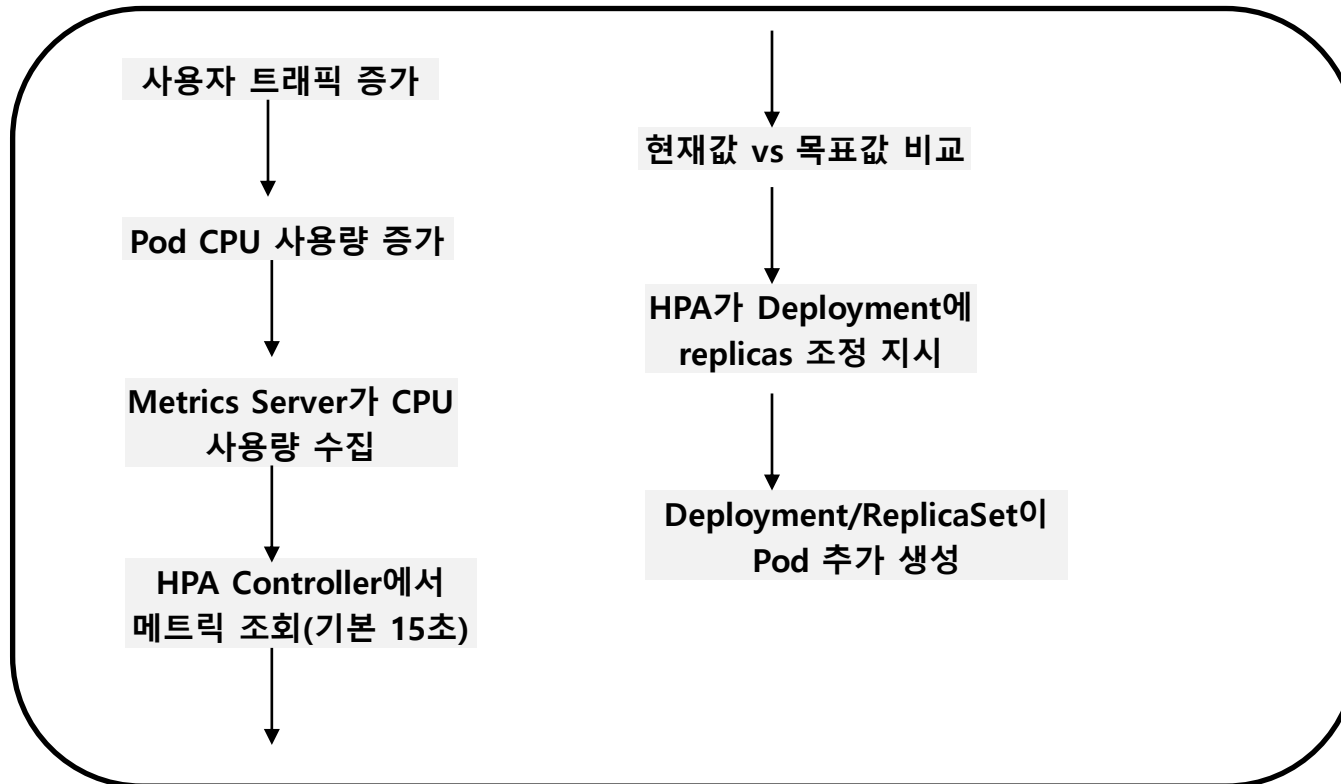
- K8s에서 Pod 개수를 자동으로 늘리거나 줄이는 수평 확장(Horizontal Scaling)을 담당하는 리소스이다.
 - 트래픽이 많아지면 Pod 개수를 자동으로 증가시키고
 - 트래픽이 줄어들면 Pod 개수를 자동으로 감소시킨다.
- <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

■ 구성요소

- **Metrics Server**
 - 기본 리소스 메트릭
 - CPU/Memory 같은 Pod 자원 사용량 제공
 - HPA가 CPU 기반으로 동작하려면 Metrics Server가 필요
- **HPA**
 - 목표치(target)를 기준으로 replicas를 늘릴지/줄일지 판단
 - 목표치 기준
 - Percentage value : 퍼센트(%) 이용
 - Raw value : 절대값 이용
- **Deployment**
 - 실제 Pod를 생성하고 replicas 관리

6.1 HPA (Horizontal Pod Autoscaler)

■ 전체 흐름 (CPU 기반)



6.1 HPA (Horizontal Pod Autoscaler)

■ 실습

```
# hpa-autoscaler.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp-deploy
  minReplicas: 1
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50 # 50%
```

```
# hpa-autoscaler-deployment-stress.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      restartPolicy: Always
      containers:
        - name: stress
          image: polinux/stress
          args: [ "stress", "--cpu", "1", "--timeout", "120s", "--verbose" ]
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "200m"
```

6.2 Probes

■ 개념

- K8s에서는 Pod가 어떻게 동작하고 있는지 더 잘 파악하기 위해서 Probe를 사용한다.
- <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

■ 종류 3가지

- **startup probe**
 - 컨테이너 어플리케이션이 실제로 시작되었는지 확인하기 위해서 사용
- **liveness probe**
 - 컨테이너가 살아있는지 확인하기 위해서 사용
- **readiness probe**
 - 트래픽 받을 준비가 되었는지 체크하기 위해서 사용
- 이 모든 probe는 kubelet이 Pod 내부에서 실행 중인 워크로드의 상태를 이해하기 위해 사용된다.

■ 동작 상태 확인 방법

- 명령어 (Command) 실행
- HTTP 요청
- TCP 요청
- gRPC 요청

6.2 Probes

■ Liveness Probes

- 컨테이너가 살아있는지 확인하기 위해서 사용된다.
- 실패하면 자동으로 재시작 된다.
- ex. Deadlock, 무한루프, 내부 오류 등
- 예를 들어, 어플리케이션이 실행 중이지만 실제로는 작업을 진행하지 못하는 데드락(deadlock)상태를
- 감지할 수 있고, 이러한 상태에서 컨테이너를 재시작하면 버그가 있더라도 어플리케이션의 가용성을
- 더 높일 수 있게 된다.

■ Readiness Probes

- 트래픽을 받을 준비가 되었는지 체크하기 위해서 사용된다.
- 실패하면 Pod는 살아있지만 Service 트래픽은 처리안됨 (즉 로드밸런싱에서 제외됨)
- ex. 앱 초기화 중, DB 연결 대기, 캐시 로딩 중

■ Startup Probes

- 컨테이너 어플리케이션이 실제로 시작되었는지 확인하기 위해서 사용된다.
- 즉, 시작 시간이 오래 걸리는 앱을 위한 Probe에서 매우 유용하다.
- 이 Probe가 성공하기 전까지는 Liveness Probe와 Readiness Probe가 동작되지 않는다.
- ex. SpringBoot, AI 모델 로딩, 대용량 초기화 등

6.2 Probes

■ Liveness vs Readiness

항목	Liveness	Readiness
목적	어플리케이션이 살아있는지 확인	요청을 받을 준비가 되어있는지 확인
실패 시	자동으로 재시작	트래픽에서 제외
서비스 영향	매우 큼	로드밸런싱에서만 영향

■ 실무 기준 사용 예

상황	추천 Probe
웹 서버	Liveness + Readiness
SpringBoot	Startup + Liveness
DB 연결	Readiness
AI 모델 로딩	Startup

6.2 Probes

■ 주요 옵션

옵션	설명	기본값
<code>initialDelaySeconds</code>	시작 후 대기시간	0 초
<code>periodSeconds</code>	검사 주기	10 초
<code>timeoutSeconds</code>	응답 제한 시간	1 초
<code>failureThreshold</code>	실패 횟수	-
<code>successThreshold</code>	성공 기준	-

■ 샘플

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
    - name: liveness
      image: registry.k8s.io/e2e-test-images/agnhost:2.40
      args:
        - liveness
```

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 2
  timeoutSeconds: 5
  failureThreshold: 5
```

6.2 Probes

■ 실습

```
# probes-ok-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: probe-ok-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: probe-ok
  template:
    metadata:
      labels:
        app: probe-ok
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          startupProbe:
            httpGet:
              path: /      # nginx 루트 경로 (정상 응답 200)
              port: 80
            periodSeconds: 5    # 5초마다 체크
            timeoutSeconds: 2
            failureThreshold: 12 # 최대 60초(5*12) 동안 부팅 기다림
```

```
readinessProbe:
  httpGet:
    path: /
    port: 80
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3
```

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```


6.2 Probes

■ 실습

```
---
apiVersion: v1
kind: Service
metadata:
  name: probe-ok-svc
spec:
  selector:
    app: probe-ok
  ports:
    - name: http
      port: 80          # Service Port
      targetPort: 80    # Container Port
      nodePort: 30080
  type:
    NodePort
```

6.3 Init Container

■ 개념

- Pod가 시작되기 전에 먼저 실행되는 특별한 컨테이너이다.
- <https://kubernetes.io/docs/concepts/workloads/pods/init-containers/>

■ 주요 용도

- DB 준비 확인
- Config 파일 생성
- Secret 다운로드
- Volume 초기 데이터 생성
- 외부 API 준비 체크

■ 동작 흐름

Init Container 1 실행



완료해야 다음 단계 진행

Init Container 2 실행



모두 성공해야 진행

메인 Container 시작

6.3 Init Container

■ 주요 특징

- 순차 실행 (Sequential Execution)
 - 여러 개 정의 가능하고 반드시 순서대로 실행됨
 - 하나라도 실패하면 다음 단계 진행 안됨
- 완전히 분리된 환경
 - 메인 Container와 분리된 환경
 - 이미지 / 명령어 / tool 모두 달라도 가능
- 종료되면 자동으로 제거
- Volume 공유 가능
 - 메인 Container와 Volume 공유

■ 일반 Container 비교

구분	Init Container	일반 Container
실행 시점	Pod 시작 전	Pod 시작 후
실행 순서	순차 실행	동시에 실행
지속 여부	완료 후 종료	계속 실행
목적	준비 작업	서비스 실행
실패 영향	Pod 시작 안됨	컨테이너만 재시작

6.3 Init Container

■ 실습

```
# init-Container-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: init-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: init-demo

  template:
    metadata:
      labels:
        app: init-demo

    spec:
      # Volume 정의 (Init ↔ Main 공유)
      volumes:
        - name: shared-volume
          emptyDir: {}
```

```
# Init Container
initContainers:
  - name: init-write-file
    image: busybox
    command:
      - sh
      - -c
      - |
        echo "[init] Writing index.html"
        echo "Hello Init Container" > /data/index.html
    volumeMounts:
      - name: shared-volume
        mountPath: /data

# Main Container
containers:
  - name: nginx
    image: nginx
    ports:
      - containerPort: 80

    volumeMounts:
      - name: shared-volume
        mountPath: /usr/share/nginx/html
```

7. Jobs / CronJobs / StatefulSets

7.1 Job

■ 개념

- K8s에서 일회성(배치성) 작업을 실행하기 위한 리소스이다.
- <https://kubernetes.io/docs/concepts/workloads/controllers/job/>

■ 특징

- 작업(Task) 기반의 워크로드를 실행하는 K8s 컨트롤러이다.
- 하나 이상의 Pod를 생성하며, 설정된 개수만큼 Pod가 성공적으로 종료될 때까지 계속 실행을 재시도 한다.
 - backoffLimit 설정 가능
- Job을 삭제하면 Job이 생성했던 Pod들도 함께 정리(Clean up)된다.
- Job안에서 정의된 컨테이너는 restart 정책이 **Never** 와 **OnFailure** 만 지원된다. (**Always** 지원 안됨)
- Job의 설정을 변경하려면 기존 Job을 수정하는 것이 아니라 삭제 후 다시 생성해야 된다.

■ Job Types (Job 유형)

- Multiple Parallel Jobs (다중 병렬 Job)
 - 여러 개의 Job(Pod)을 동시에 실행하여 가능한 경우 작업을 더 빠르게 처리하도록 함
 - 일반적으로 completions 와 같이 사용됨
- Fixed Completion Count (고정 완료 횟수를 가진 Job)
 - 동일한 작업을 정해진 횟수만큼 반복하고 성공적으로 완료되면 작업이 종료된다.
 - 매 회마다 새로운 Pod가 새로 생성되어 처리 된다. (병렬처리 가능)
- Non-parallel Jobs (비병렬 Job)
 - 하나의 작업을 단독으로 실행하는 Job으로서, 만약 실패 시 새로운 Pod가 다시 생성된다.
 - Pod가 한 번 성공적으로 종료되면 그 Job은 완료된 것으로 간주한다.

7.1 Job

■ 핵심 속성

속성	설명
completions	성공해야 할 작업 개수
parallelism	동시에 실행할 Pod 개수
backoffLimit	실패 재시도 횟수
restartPolicy	Never / OnFailure (Always 지원 안됨)

7.1 Job

■ 실습1- Non-parallel

```
# job-Non-parallel.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  template:
    spec:
      containers:
        - name: hello
          image: busybox
          command:
            - sh
            - -c
            - |
              echo "Hello Kubernetes Job"
              sleep 5
              echo "Job Finished"
      restartPolicy: Never # Job은 Always 사용 불가
      backoffLimit: 3      # 실패 시 최대 재시도 횟수
```


7.1 Job

■ 실습1- Non-parallel

```
# job-Non-parallel.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  template:
    spec:
      containers:
        - name: hello
          image: busybox
          command:
            - sh
            - -c
            - |
              echo "Hello Kubernetes Job"
              sleep 5
              echo "Job Finished"
      restartPolicy: Never # Job은 Always 사용 불가
backoffLimit: 3          # 실패 시 최대 재시도 횟수
```

7.1 Job

■ 실습2- Fixed-Completion-Count

```
# job-fixed-Completion-Count.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: fixed-job
spec:
  completions: 5      # 성공적으로 5번 실행되면 종료
                      # 매 회마다 새로운 Pod가 생성됨
  template:
    spec:
      containers:
        - name: counter
          image: busybox
          command:
            - sh
            - -c
            - |
              echo "RUN: $(hostname)"
              sleep 5
              echo "DONE"
      restartPolicy: OnFailure
```

7.1 Job

■ 실습3- Multiple-parallel

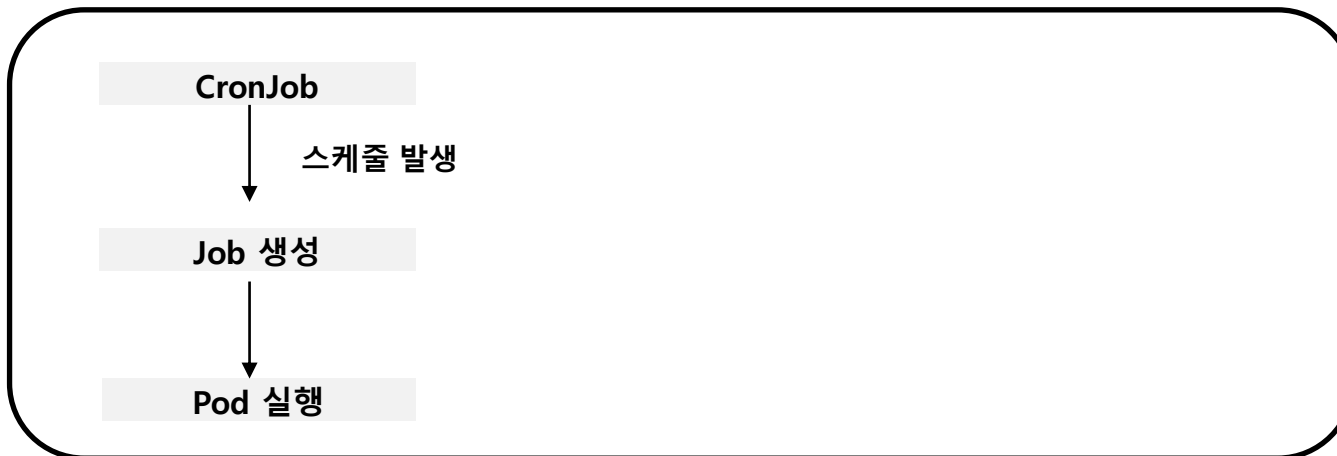
```
# job-multiple-Parallel.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel-job
spec:
  parallelism: 3      # 동시에 실행할 Pod 수(최대)
  completions: 6     # 성공해야 하는 총 횟수
  backoffLimit: 2     # 각 Pod 실패 시 재시도 횟수
  template:
    spec:
      containers:
        - name: worker
          image: busybox
          command:
            - sh
            - -c
            - |
              echo "START: $(date) / POD: $(hostname)"
              sleep 15
              echo "DONE : $(date) / POD: $(hostname)"
          restartPolicy: OnFailure
```

7.2 CronJobs

■ 개념

- Job을 정해진 시간마다 자동 실행하는 리소스이다.
- 리눅스의 cron과 동일한 개념이다.
 - 매일 새벽 백업
 - 로그 정리
 - 데이터 수집
 - 배치 리포트 생성
- <https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>

■ 동작 흐름



7.2 CronJobs

■ 주요 속성

속성	설명
schedule	실행 시간
concurrencyPolicy	동시에 실행 허용 여부 Allow: 중복 실행 허용 Forbid: 이전 Job 끝나야 실행 Replace: 기존 Job 종료 후 실행
successfulJobsHistoryLimit	성공 Job 보관 개수
failedJobsHistoryLimit	실패 Job 보관 개수

7.2 CronJobs

■ 실습

```
# job-cronJobs.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: simple-cronjob
spec:
  schedule: "*/1 * * * *" # 1분마다 실행
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox
              command: ["sh", "-c", "date"]
          restartPolicy: OnFailure
```

7.3 StatefulSets

■ 개념

- 상태(State)를 가지는 어플리케이션을 관리하기 위해 설계된 Pod 컨트롤러이다.
- <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/>
- <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>

■ 필요한 이유

- 일반적으로 Deployment를 사용할 때는 어플리케이션을 Stateless 방식으로 설계한다.
 - 즉, 특정 Pod에 데이터나 상태가 저장되지 않아야 된다.
- 따라서 Pod는 언제든지 삭제되고 다시 생성될 수 있으며, 이름과 IP가 일정하게 유지되지 않는다.
 - Deployment 환경에서는 Pod의 고정된 식별자가 보장되지 않는다.
- 하지만 어떤 경우에는 실행할 때마다 고정되어 항상 사용할 수 있는 연결된 볼륨이 필요할 수도 있다.
 - 즉, 다른 리소스에 정의된 PVC에 의존하지 않고 개별적으로 관리하고 싶을 수도 있다.

7.3 StatefulSets

■ 특징

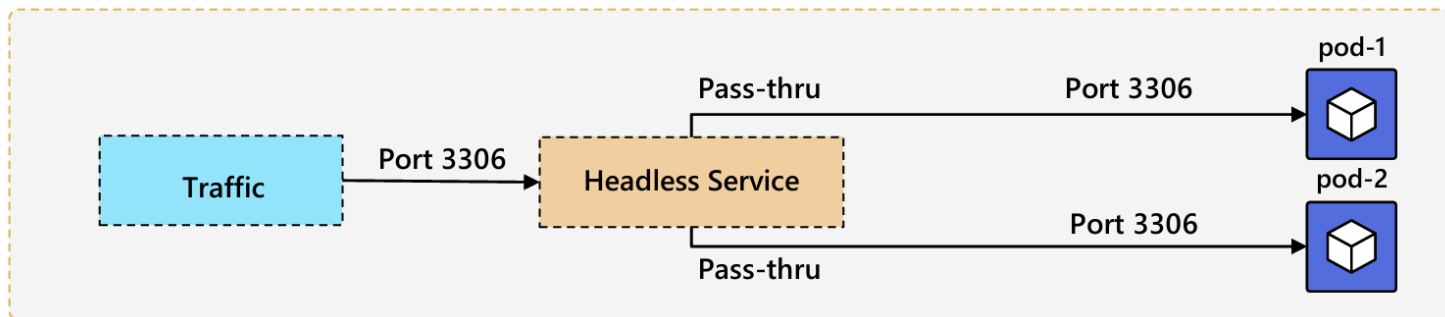
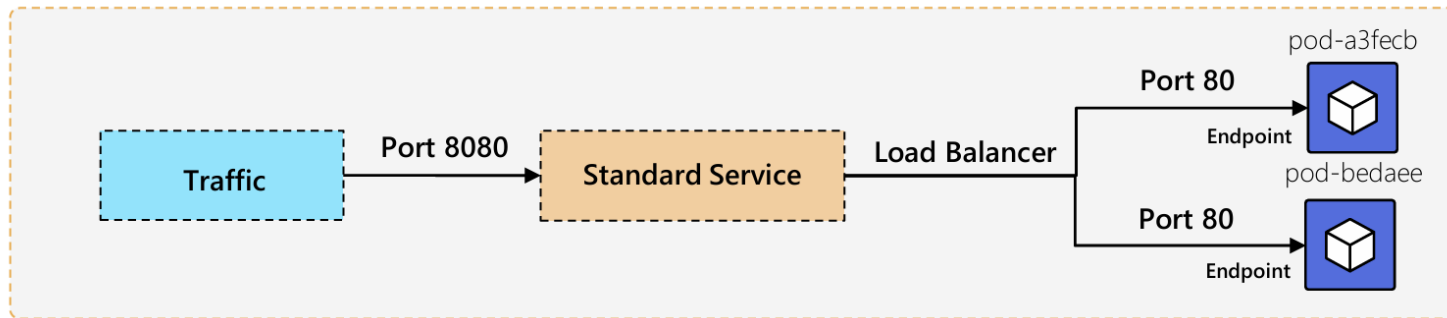
- Deployment와 달리 StatefulSet은 각 Pod에 고유하고 고정된(sticky) 이름을 유지한다.
 - Pod가 재시작되거나 재스케줄링 되어도 동일한 이름과 정체성을 유지한다.
 - ex. app-0, app-1, app-2
- Pod들은 동일한 스펙으로 생성되지만 서로 교체 가능한 존재는 아니다.
 - 각 Pod는 재생성 이후에도 유지되는 고유한 영구 식별자(persistent identifier)를 가진다.
- Pod는 순서대로 실행되며, 0부터 시작하는 연속된 번호 (0,1,....., N-1)가 부여된다.
 - ex. pod-0 실행하고 이후 pod-1, pod-2 순서로 실행
- Headless Service가 필요하다 (**ClusterIP: None**)

■ Deployment vs StatefulSet

구분	Deployment	StatefulSet
Pod 이름	랜덤	고정 (index 있음)
Pod 순서	없음	있음
저장소	공유 가능	Pod 별 고정 PV
Scale	동시에	순차적
Service	일반 Service	Headless Service (Cluster: None)
용도	Stateless	Stateful

7.3 StatefulSets

■ 일반 Service vs Headless Service



7.3 StatefulSets

■ 실습 1- basic

```
# statefulset-basic.yaml
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: sts-demo
---
```

```
# Headless Service (StatefulSet 필수)
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-hl
  namespace: sts-demo
spec:
  clusterIP: None
  selector:
    app: web
  ports:
    - port: 80
```

```
---
```

```
# StatefulSet
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
  namespace: sts-demo
spec:
  serviceName: web-hl      # Headless Service 이름
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
```

7.3 StatefulSets

■ 실습 1- basic

```
---
# DNS 테스트용 Pod
apiVersion: v1
kind: Pod
metadata:
  name: dns-client
  namespace: sts-demo
spec:
  containers:
    - name: busybox
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
```

7.3 StatefulSets

■ 실습 2- PV

```
# statefulset-pv-retain.yaml
```

```
apiVersion: v1
```

```
kind: Namespace
```

```
metadata:
```

```
  name: sts-pv-demo
```

```
---
```

```
# Headless Service
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: app-hl
```

```
  namespace: sts-pv-demo
```

```
spec:
```

```
  clusterIP: None
```

```
  selector:
```

```
    app: app
```

```
  ports:
```

```
    - name: dummy
```

```
      port: 80
```

```
      targetPort: 80
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: StatefulSet
```

```
metadata:
```

```
  name: app
```

```
  namespace: sts-pv-demo
```

```
spec:
```

```
  serviceName: app-hl
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: app
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: app
```

```
    spec:
```

```
      containers:
```

```
        - name: busybox
```

```
          image: busybox:1.36
```

```
          command: ["sh", "-c", "sleep 3600"]
```

```
        volumeMounts:
```

```
          - name: data
```

```
            mountPath: /data
```

```
      volumeClaimTemplates:
```

```
        - metadata:
```

```
          name: data
```

```
        spec:
```

```
          accessModes: ["ReadWriteOnce"]
```

```
          resources:
```

```
            requests:
```

```
              storage: 1Gi
```

수고하셨습니다.